面向三维重建的点云重采样技术

摘　要：随着人们对图形图像应用需求的高速发展，获取简单、包含丰富信息的三维点云数据得到了越来越广泛的关注。由于受到采集设备和周围环境的影响，直接扫描产生的点云通常存在冗余、噪声、不均匀等特性。因此，将点云重采样获得一个干净、均匀、特征自适应的结果，这对于三维重建和各种图形学应用变得十分重要。本文设计了一个对点云进行特征信息提取的多级跃层连接模块 (Cascaded Skipping Connection Module, CSCM)，并构建了一个深度神经网络进行点云分布优化，根据点的位置坐标和法向信息进行基于曲率的降采样。多级跃层连接模块充分利用了点云的局部特征，又兼顾考虑了全局分布特点。而联合损失函数的设计，则保障了结果尽可能与原始点云的特征相似性以及局部区域的均匀性。网络优化点云并生成法向信息，借助法向获得准确的曲率后，通过一个基于曲率的采样策略，得到了特征自适应的采样结果。实验证明了所使用的重采样方法的有效性，并且优于其他对比方法，能够实现高质量的三维重建。

关键词：点云重采样；深度学习；点云重建

Point Cloud Resampling Technology for 3D Reconstruction

ABSTRACT: With the rapid development of the graphics and image applications, obtaining simple and informative 3D point cloud data has received more and more attention. Due to the effects of acquisition equipment and surrounding environment, the point clouds obtained by scanning often have the characteristics of redundancy, noise and non-uniformity. Therefore, resampling point clouds to reach a clean, uniform and featured goal becomes very important for 3D reconstruction and various graphics applications. In this paper, a cascaded skipping connection module (CSCM) is proposed, which can be designed for point cloud feature extraction, and a deep neural network is constructed to optimize the distribution. Downsampling based on curvature is carried out according to the locations and normals of points. CSCM makes full use of the local features and also takes into account the global distribution features. The design of the joint loss function encourages the similarity between the results and the original point clouds and the uniformity of the local regions. After generating the normal information by the network and estimating the curvature, a curvature-based sampling strategy is used to acquire the feature-adaptive sampling results. Experiments demonstrate the effectiveness of the resampling technology, which is superior to other comparison and can achieve high-quality 3D reconstruction.

KEYWORDS: Point cloud resampling; Deep learning; Point cloud reconstruction

目 录

摘 要 …… I
ABSTRACT …… II
目 录 …… III
第一章 绪论 …… 1
1.1 研究背景及意义 …… 1
1.2 国内外研究现状 …… 2
1.2.1 传统几何方法 …… 2
1.2.2 应用于点云的主干深度神经网络 …… 3
1.2.3 点云重采样相关的深度学习方法 …… 4
1.3 本文主要内容及创新点 …… 5
1.4 本文组织结构 …… 6
第二章 开发平台及工具 …… 7
2.1 开发平台 …… 7
2.2 深度学习框架 PyTorch …… 7
2.3 三维数据处理库 Open3D …… 8
第三章 面向三维重建的点云重采样设计 …… 9
3.1 问题定义 …… 9
3.2 相关知识简介 …… 9
3.2.1 点云的基本特性 …… 9
3.2.2 多层感知机与点云 …… 9
3.2.3 对称函数处理 …… 10
3.3 方案设计 …… 11
3.3.1 总体架构 …… 11
3.3.2 多级跃层连接模块 …… 13
3.3.3 基于曲率的采样 …… 14
第四章 技术具体实现 …… 15
4.1 网络结构的关键实现 …… 15
4.1.1 多级跃层连接模块实现 …… 15
4.1.2 特征提取实现 …… 16
4.2 数据集 …… 16
4.3 损失函数 …… 17

第五章 实验结果 …… 19
5.1 对比其他方法 …… 19
5.1.1 评价指标 …… 19
5.1.2 定量比较 …… 19
5.1.3 定性比较 …… 20
5.2 三维重建 …… 22
5.3 消融实验 …… 24
5.4 参数的影响 …… 25
5.5 健壮性分析 …… 26
第六章 总结与展望 …… 29
6.1 本文研究总结 …… 29
6.2 未来研究展望 …… 29
参考文献 …… 31
致 谢 …… 35

第一章 绪论

1.1 研究背景及意义

随着生产生活场景的丰富和信息技术的革新，人们对于图形图像的应用需求高速增加。从每天生活中常见的手机拍照、笔记本办公，到各种高新技术产业的无人机、无人车、智慧医疗场景等等，人们随时随地接收着各种类型格式的图像信息，而对于这些图形图像信息需求多元化的同时，计算机视觉与计算机图形学也由此蓬勃发展开来。与二维的图片、视频等数据相比，三维数据能呈现出更加立体直观的视觉效果，所包含的信息更加丰富，在计算机视觉与图形学中发挥的作用也更加巨大，处理三维格式的数据也更为困难。

常见的三维数据类型有点云 (Point Clouds)、网格 (Meshes)、体素 (Voxel Grids)、多视角表示 (Multi-view Representations) 等。其中，点云数据由激光雷达、深度相机等采集设备直接扫描现实的物体和场景获取，每个点由空间坐标 (X，Y，Z) 表示，切实反映出所扫描的场景，因此其获取简单、表达灵活、操作多样的优势被人们所青睐，近几年，它已经在不同的场合下得到了广泛的使用[1-3]。但是，点云中每个点是离散无序的，并不具有连接或者拓扑关系，无法生成连续的平面，而这些不足又可以使用三维网格来填补[4, 5]。网格中带有连接关系的顶点可以组成一系列三角面片，这些三角面片可以渲染出任意连续的曲面，得到紧凑平滑的场景，由此在计算机图形学中大量使用，不仅用于动画、游戏渲染，也可在虚拟现实、增强现实技术中发挥作用。既然点云数据是在三维场景中容易捕捉，而网格数据在渲染中使用较为方便，那么，如何将点云转化为紧凑的网格表示，就显得十分关键。

三维重建，就是根据点云等可以直接捕捉的数据形式，重建出连续的三维数据模型。将点云转化成网格，就是三维重建中的重要子问题。但由于硬件设备的误差、物体的光照反射、场景的干扰遮挡等原因，扫描获取的点云数据经常有丢失、冗余、噪声、不均匀等特性，直接重建会影响质量。因此，原始点云就需要先借助点云的重采样技术进行处理后，再进行三维表面重建，才能对重建模型质量与效果的提升带来帮助。

输入一个原始的扫描点云，适当地变形重构点的形状分布，改变点的数目，减少噪声和冗余，以提高点云的质量，并为后续的各种应用提供质量的保障，这个过程就是点云的重采样技术。点云往往通过点之间不同的分布和密度来包含它所代表物体的各种显著特征，在重采样处理中，需要尽可能地保留这些特征，结果也要尽可能地接近输入点云。

大部分重采样工作主要是对原始密集的点云去除冗余的点，即进行点云降采样。在满足将原始采样点云简化为点数更少的要求的同时，更需要使其具有更低的噪声和更均匀的局部分布，以及所得到的点云在特征丰富的区域（如边缘和棱角处）保留较多的点数，而在光滑平坦的地方保留较少的点数，以实现特征的自适应性。因此，对于点云重采样技术的探索在三维表面重建以及后续阶段的使用具有重要研究与应用价值。

1.2 国内外研究现状

在点云数据发展的早期，研究者们大多使用数学上几何的方法来计算并标识原始点云特征，根据几何特征进行不断迭代优化得到重建结果，这些方法往往只能针对特定的物体或者场景发挥作用。而在人工智能技术快速发展的时代背景下，深度学习技术广泛应用于计算机图形图像研究与自然语言处理，且均显露出它在特征提取方向的强大优势。于是深度学习方法也开始逐渐地被应用于点云分类等任务，点云的重建与采样技术也得益于这些网络的研究。

1.2.1 传统几何方法

由于三维扫描设备的普遍使用，在早期产生了大量密集、嘈杂、冗余的点集，这对于计算机图形学上模型的表示造成了一定的不便。对于不同密度的点云，Alexa 等人[6]受到微分几何的启发，使用移动最小二乘法 (Moving Least Squares, MLS)，从局部空间不断逼近曲面，从而调整点之间的间距来改变点云密度，获得光滑的重建曲面。尽管该方法在当时对于点集数据表面拟合十分有力，但在点云的高曲率区域相当地不稳定。

Lipman 等人[7]开发了用于几何物体重建的无参局部最优投影算子 LOP (Locally Optimal Projection)，它源自多变量 L_1 中值办法，可以很好地处理原始数据，而与点云中每个点具体的局部几何参数无关，因此对直接扫描产生的噪声点和离群点有一定程度上的鲁棒性。然而，这种方法存在高计算成本的缺点，并且不能很好地保留几何特征。当输入点的分布高度不均匀时，它可能会无法正常处理数据。而 Huang 等人[8]将点云在不同局部区域的密度值加权进 LOP，改进为 WLOP (Weighted Locally Optimal Projection)算法，较好地处理了具有非均匀采样的点集。他们还提出了一种基于优先级驱动的法线传播和方向敏感的 PCA 法线估计方法，较好地估计点法线，提供了点集的局部表面近似与内外方向，这对于表面重建至关重要。FLOP (Feature-preserving Locally Optimal Projection) [9]借助无方向的法线估计，基于双边加权的 LOP，同时考虑物体空间和几何信息的特征保持。同时，通过使用核密度估计 (Kernel Density Estimate, KDE) 的随机采样来大大加速 FLOP，获得有效忠实的重建曲面。但 FLOP 依赖于均匀分布的初始点，对数据丢失、噪声等的鲁棒性较低，在处理严重嘈杂的表面时可能会造成结果的不平滑。

针对由 PCA[10]计算出法线的有噪声点云，在其边缘附近的一些点会错误地分配法线方向，这可能会在重建的表面上造成可见的伪影。EAR (Edge-Aware Resampling) 算法[11]为了解决这个问题，以边缘敏感的方式重新采样噪声点集。它首先对远离边缘部分重新采样，这些点都有较为正确可靠的法线，并据此逐渐地向边缘靠近并再次采样点集。因此，从可靠的区域延伸到不太可靠的边缘，避开了不可靠区域对噪声敏感的法向信息，实现了对噪声和异常值的鲁棒性。之后，Preiner 等人[12]继续改进优化 LOP，设计了连续的投影算子 CLOP (Continuous LOP) 进行加速处理。

传统几何方法基于空间中的数据点定义几何体，通过每个点的法向、曲率、密度等基本几何性质进行重采样，最终生成质量较高且局部分布较为均匀的曲面，但是这些方法均耗时长，鲁棒性不高。另外，也有基于谱分析的流形重采样[13]、基于图形轮廓的重采样[14]等算法，但这些传统方法都需要法向、曲率等额外的点云特征输入，这加大了存储开销，也不方便数据的采集。

1.2.2 应用于点云的主干深度神经网络

由于点云的无序性、点之间的相关性、置换不变性[15]，在二维平面图像上十分流行的卷积神经网络 (Convolutional Neural Networks, CNN) 不可以直接使用，研究者们往往将点云投影成二维平面的不同视角[16]，或者划分为具有依赖关系的体素[17]处理。这一困难直到2017 年，PointNet 由 Qi 等人提出[15]，将点云直接运用到深度学习上来。PointNet 通过多层感知机 (Multi-layer Perceptrons, MLPs) 将点的三维空间坐标映射至高维，并通过一个最大池的对称函数聚合得到全局特征，精妙地解决了点云无序和不变性所带来的麻烦。作为第一个直接适用于原始点云的网络，PointNet 迅速流行起来，并成功地用于点云的基本特征提取，在点云分类、部件分割、场景语义分析中效果显著。

然而，PointNet 并未考虑点与点之间的局部空间关系，原作者继续提出了分层神经网络 PointNet++[18]，通过在输入点集上进行分层，每层又由三个关键层组成：采样层、组合层和特征提取层。网络中串联起多个这样的层次，递归地应用 PointNet 实现细节捕获和鲁棒性，PointNet++就如二维 CNN 一样从浅到深地得到语义特征。最终每个点的特征不仅包含它自己的特点，而且含有自己与周围近邻点之间的关系。在每一层选取局部点组合时，由于点云在每个区域的密度一般是不同的，这会导致每层局部区域包含的点数不一致，从而对后续网络的学习造成影响。对此，多尺度分组 (Multi-scale Grouping, MSG)以及多分辨率分组 (Multi-resolution Grouping, MRG) 的两个策略产生，解决了均匀性不同的采样问题，保证了更加良好的特征提取效果。PointNet++在分类和分割问题上准确率远超 PointNet，尤其是对于大量稀疏点云分类时使用多尺度分组特征策略，大大提高了分类精度。

后续工作中，PointCNN[19]将图像 CNNs 做到了推广，提出用点学习 X 变换 (X-transformation) ，将输入“汇集”转化后得到更少数目的点，在每一点上都会包含更多的特征，然后对变换后的特征再应用经典的卷积算子。该网络在点云的分类和分割任务、甚至对普通二维图像的分类上都取得了显著效果，弥补了直接对点云用 CNNs 会导致形状信息丢失和点排序变动的缺点。为了进一步捕获点和点的几何关系和局部领域特征，DGCNN[20]设计出边卷积 (EdgeConv) 模块，它既能够捕捉局部几何结构，又能维持点云的排列不变。EdgeConv 不直接生成点的特征，而是生成描述点与其邻居之间关系的边特征，再将这些边特征聚合得到中心点的表征。在特征空间中，不断重新计算每个点在每层的近邻，动态更新局部领域图结构，从而获取局部几何结构以及全局语义特征。

另外，受 PointNet 在点云分类上的成功启发，近年来许多其他点云应用也不断应运而

生，例如点云检索[21]、重建[22]、配准[23]和目标检测[24]等等。尽管它们有不同的任务目的，但这些网络设计都借鉴了上述基本的网络架构，并针对性地加以集成和改进。

1.2.3 点云重采样相关的深度学习方法

与传统数学上的几何方法不同，利用深度学习技术进行重采样往往与分类、检索等下游任务有关，针对特定的任务功能来选择特征。Yang 等人[25]采用 Gumbel Softmax 层来聚合高层特征以实现后续更高的分类精度，Nezhadar 等人[26]使用了最大池化层临界点唤醒的方式寻找对分类任务最关键的点，但他们都只是选择点来标识特征用于分类。

首次对点云实现降采样的 S-NET (Simplification Network) [27]，是在 PointNet 框架上获得全局特征，再由全连接层获得较少数目的点。在 S-Net 网络后面跟随一个预训练的任务网络，这样就可以达到端到端训练的目的。作者提出了采样损失和任务损失，平衡制约了简化过程中既要与原始形状相似又要将其优化到后续任务的矛盾。S-NET 生成的简化点云不能保证是源自输入点云的，于是在推理时执行一个后处理的匹配步骤以获得输入的一个采样子集。与经典的最远点采样 (Farthest Point Sampling, FPS) 和随机采样等非学习方法相比，这种方法提高了采样点云的应用性能。Lang 等人[28]提出了 SampleNet，对 S-NET 的工作进行了修改。通过在训练时向匹配步骤引入软投影 (Soft Projection) 的操作，将采样集的点替换为最近邻点的加权值。该操作采用模拟退火策略，促进每个简化点是用于后续任务的最佳输入点。软投影的提出使得匹配变得可微，能够通过神经网络传播梯度，提高了采样点的性能，在分类、配准、重建任务中发挥重要作用。

然而，S-Net 和 SampleNet 都直接从全局特征到降采样坐标，不考虑采样点之间的空间相关性。空间中互相接近的点倾向于由相同的下采样点表示，这一局部联系在降采样中起着重要作用。同时，多任务驱动的目标导致它们都是语义相关的，即面向分类等语义信息处理的问题，而非面向三维重建，因此无法生成高质量且均匀分布的简化点云。

与降采样相反，上采样也是一个对三维重建意义重大的任务。上采样可以被视为图像超分辨率的 3D 版本，或者是下采样的逆过程。由于设备直接捕获的点云并非都是稠密的，有时会因为过于稀疏而造成部分细节的损失，因此需要先对稀疏的低质量点云进行上采样，为重建高质量曲面做准备。因此，上采样作为一项生成任务，需要丰富的信息特征扩展，并且可以通过实际真实点云进行训练。

PU-Net (Point Upsampling Network) [29]是上采样领域的先驱代表。无序不规则的点云结构导致不能像图像超分辨率一样简单地插值处理，PU-Net 基于点云提取后的小块，借鉴 PointNet++网络构件进行点特征的学习，通过设计的两组特征扩展卷积实现点云数目的增加，达到上采样目的。该方法中联合损失的设计引导最后的结果分布更均匀，离物体的底层表面更近。在其他的深度学习上采样模型中，包括引入视觉上做生成任务的对抗网络来训练上采样点集的 PU-GAN (Point Upsampling Generative Adversarial Network) [30]、借助几何特征和传统的参数化表面理论的 PUGeo-Net (Point Upsampling Geometric-centric Network)

[31]等等方法，它们均是对输入点云提取特征或者几何拟合，再生成增加点数的目标点云，但都强调生成结果的均匀性，对于特征的自适应没有充分考虑。

1.3 本文主要内容及创新点

针对三维重建的点云重采样，本研究是基于深度学习的，相比单一的传统几何优化方法速度更快，且并不需要额外的输入或者依赖。同时，本方法综合几何采样的特点，关注于局部分布均匀、特征自适应、保真度高的点云，从而便于直接被后续的表面重建任务使用，获得高质量的网格模型，这是此前面向分类分割等语义处理任务的重采样深度神经网络不具有的功能。本文的点云重采样方法流程图如图 1-1 所示。

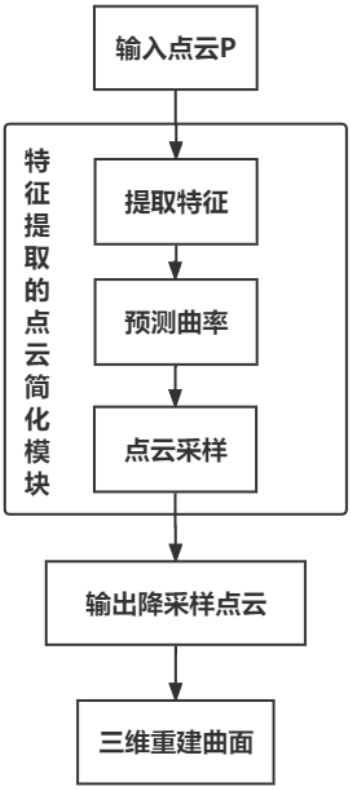


图 1-1 点云重采样流程图

该重采样流程主要由点云特征提取、预测曲率和点云采样三个部分组成点云的简化模块。在点云特征提取中，通过设计的一个深度神经网络结构提取出点云的特征，由点云特征获得优化后的新的点云分布。同时，每个点比较精确的法向信息也可以根据网络得到，而非传统方法上的几何拟合估计。在预测曲率和点云采样模块，根据每个点的近邻域信息和法线，获得预测的曲率。点云中曲率愈大的地方，重采样时选择的点数愈多，反之采样点个数愈少，从而获得重采样点云的输出结果。结合该点云和法向信息，就可以通过泊松重建[32]获取高质量的三维表面。

本文所研究设计的网络使用已有的经典三维模型库进行训练，同时加入较为复杂的扫

描点云数据集。在大量数据集上的实验对比与证明，本文的点云重采样技术能在较小数目的点采样中很好地保持局部分布的均匀性，并保留更多的点在特征丰富的区域，更少的点在平坦区域，实现特征的自适应性。

围绕以上内容，本文的创新点主要体现如下：

（1）在对点云进行特征提取时，设计了多级跃层连接模块 (Cascaded Skipping Connection Module, CSCM)。通过一系列的级联与复制操作，更好地捕获了点云的全局特征，同时又充分考虑了点与点之间的局部特点。

（2）本研究所构建的深度神经网络能够优化点云分布和直接预测点的法向信息，通过该法向就可以根据曲率进行特征保持的采样，这与使用传统 PCA 法线估计[10]方法获取法向不同，减少了对于几何分布的依赖和拟合误差。

1.4 本文组织结构

本文一共包含五章内容：

第一章，绪论。本章概括说明了点云重采样领域的研究背景及意义，介绍了国内外对三维点云数据处理的研究现状，并简要解释了其贡献和不足，最后则是关于论文的主要内容、创新点与组织结构。

第二章，开发平台及工具。本章介绍了重采样研究的实验平台、深度学习框架与支持三维数据的库。

第三章，面向三维重建的点云重采样设计。本章陈述了研究问题，简单介绍了点云的性质与基本的点云深度学习要素，详细解释了本文所设计的面向三维重建的点云简化模块和重采样流程。

第四章，技术具体实现。本章给出了深度学习网络设计的关键实现代码，同时对所使用数据集的介绍和处理、训练时损失函数的组成和设计也有明确的阐述。

第五章，实验结果。本章指出了点云采样评价的不同标准，并通过一系列的实验，对比不同重采样方法，验证了本文方法的有效性。

第六章，总结与展望。本章回顾总结了所设计的重采样技术，指出了采样处理中仍然存在的不足，展望未来可能需要进一步优化改进的方向。

第二章 开发平台及工具

2.1 开发平台

本文设计的点云重采样技术中，深度学习网络模块的硬件平台服务器 CPU 型号为 i9-9900X，GPU 型号为 NVIDIA GeForce RTX 2080 Ti，该计算机的软件操作系统为 Ubuntu 18.04 LTS 版本。开发程序使用 Python 3.7.9 版本语言，并在 Anaconda 中建立 CUDA Toolkit 10.1 环境，安装 PyTorch 1.5.1 版本以及相应的模块库。网络使用 Adam 优化方法[33]迭代更新，它的学习率选取 0.001，β_1和β_2分别选取 0.9 和 0.999。

在点云重采样的其他处理模块，使用一台笔记本电脑进行处理。硬件上 CPU 型号为 i7-8565U，软件上操作系统为 Windows 10（64 位），并在 JetBrains PyCharm 5.0.3 集成开发环境中运行 Python 语言和安装相应的模块库。

2.2 深度学习框架 PyTorch

在进行深度学习时，选取一个适当的框架非常关键。而 PyTorch 就是一个优秀的动态图框架，快速而且容易部署，适合研究学习和实现各种算法。

PyTorch 最开始来源于 2002 年创建的 Torch 框架[34]。它使用 Lua 作为编程接口，但因为 Lua 的用户群体很小，即使它在实现深度神经网络方面有一定的效率，却一直不大受欢迎。而另一方面，由于 Python 编程语言的方便性和完善的生态应用，其在计算机领域逐渐普及开来。考虑到这一点，2017 年的 Facebook 团队重新设计了所有基于 Torch 张量的运算模块，并将其开源到 GitHub 上，于是 PyTorch 正式诞生。研究人员们还在其中增加了自动求导的先进操作，引入了一个强大的动态图架构。由于 PyTorch 的简单性和易用性，这一工具迅速流行到各式各样的人工智能领域，并成为了最受欢迎的深度学习工具之一。

在 2018 年的 NeurIPS 会议期间，Facebook 结合 Caffe2 和 ONNX 两者模块化的优势，推出了 PyTorch 1.0 版本，使算法不再局限于设计原型，而是可以进一步高效面向生产。PyTorch 生态系统由此更加全面，模型的构建和部署更加容易。目前 PyTorch 在问世的数年里，正迅速发展更新，如今已经进行 1.12 版本的测试运行之中。

与其他知名的框架，如 TensorFlow、MXNet、Keras 和 Caffe 等，PyTorch 简洁且优雅，其编码风格与标准的 Python 代码非常相似，这使得算法与模型的各种结构和逻辑更容易被用户理解。而 PyTorch 由于其自动求导的机制，使用户更容易调试和理解模型，看清每个步骤的处理过程，不必过多强调神经网络中晦涩难懂的概念。此外，和 TensorFlow 不一样，PyTorch 的所有接口都非常容易使用，灵活程度高，具有极大的兼容性，整体上对于模型速度的表现也极为出色。

2.3 三维数据处理库 Open3D

在智能机器人技术和计算机图形学等应用中三维数据十分常见，但与其他数据类型相比，处理此类数据的软件相当麻烦。不同于二维数据可以使用几行 OpenCV 代码，有效地加载和可视化图像，三维数据的类似软件框架则较少。最初应用非常广泛的三维库为点云库 (Point Cloud Library, PCL)，但之后随着各种功能的增加，PCL 变得臃肿起来，现在基本上处于停止维护的状态。

Open3D 开源处理库[35]则可以进行三维类型数据的高效处理，目前主要使用于 C++和 Python 方式。它具有较少的依赖项，可以在不同平台上直接安装使用或者从源代码快速编译。由于其整洁的代码、一致的风格，并通过清晰的代码审查机制不断进行维护，迅速成为许多三维研究项目团队的选择。

Open3D 能够支持基本点云、体素、网格等 3D 数据结构的读写，可以进行常见的 3D 数据处理算法与数据可视化操作。同时，Open3D 中的所有数据结构都与 NumPy 原生兼容，能方便地实现 NumPy 矩阵和三维向量的转换和使用。

第三章 面向三维重建的点云重采样设计

3.1 问题定义

给定一个三维点云$P = \{p_i \in \mathbb{R}^3, i = 1,2,3,\dots,n\}$，我们希望获得一个点数更少的点云满足：$Q = \{q_i \in \mathbb{R}^3, i = 1,2,3,\dots,n\}$，其中$m \le n$。那么，从数学角度上描述这个问题，即为：

$$min\|S(P) - S(Q)\| \qquad s.t.\ Q = T(P) \tag{1}$$

$T(\cdot)$表示本文所设计的重采样方法，$S(\cdot)$表示点云所代表的表面信息。该问题的目标就是尽量使用较少点数的点云获得更接近真实表面信息的结果。那么，这样的点云结果会在边、角等特征明显的区域有更多的点，在平坦区域有更少的点，达到特征的自适应性与局部均匀性，从而实现更少的点重建出更精细的表面。

3.2 相关知识简介

3.2.1 点云的基本特性

对于点云格式的三维数据，可以表示为集合$\{p_i \in \mathbb{R}^3 | i = 1,2,3,\dots,n\}$，其中每个点$p_i$包括基本的空间位置坐标$(X, Y, Z)$以及可以额外附加的特征向量，例如颜色$(R, G, B)$、法线$(N_x, N_y, N_z)$等。不同于网格或者体素格式，点云集合具有无序性、点之间的相关性以及置换不变性[15]。

点云的无序性是指每一个点在集合中的排列都是没有顺序的，它们散乱地分布在空间中。这与二维图像中整齐有序的像素点是不一样的，这就要求处理点云的模型需要对点集数据的不同排列维持不变。

点之间的相关性说明每一个点不是孤立的，点与点之间是有相互作用的，每个点与周围的所有点都存在一定的空间距离，这些距离的远和近对局部结构的构成有十分重要的意义。处理点云的模型需要有效利用这种空间关系，对点的局部空间特征和整合形成的全局特征进行捕获。

置换不变性是指对于点集进行平移、旋转等空间变换后，这个点集所代表的物体意义仍是不变的，模型对点集的处理学习仍是同一个物体。

3.2.2 多层感知机与点云

神经网络在当前发展和运用十分广泛，也产生了很多像误差反向传播网络、长短期记忆网络、卷积神经网络等等新颖的变种设计，但是多层感知机 (Multi-layer Perceptrons, MLPs) 为最简洁的结构。

MLP 通常包含三个层次，它们依次为输入层、隐含层和输出层，位于感知机中间且占主体部分的隐含层可以按一定的顺序连接多个，而最简单经典的 MLP 则只含有一个隐含

层。神经网络每一层节点的个数表示该层的特征维度大小，每一层之间采用全连接结构，即前后全部节点都有连接。通过设置一定的网络层数和节点数目，多层感知机就可以学习到数据在多个不同维度下所包含的特征信息。

对于每一层网络具体的计算，输入向量X后，输出$Y = f(w_i X + b_i)$，权重w_i、偏置b_i和激活函数$f(\cdot)$是运算的基本要素。其中，权重表征该传递方向上概率的多少，而选用一定的偏置是为了确保不随便激活输出的值，激活函数则是扮演非线性映射的角色，控制着在特定区域内节点的输出范围，从而使得任意的非线性函数都能够由构造出的神经网络进行近似模拟，发挥其在各种非线性模型中的重要作用。具体采用何种激活函数需要依照具体情况来确定，激活函数 Sigmoid 和 Tanh 是在深度网络中研究人员们十分流行选择的两个，其中函数 Tanh 是在函数 Sigmoid 基础上变化而来的。ReLU、Leaky ReLU 和 ELU 等激活函数也逐渐被流行使用。当输入的信号数值低于零时，ReLU 函数会输出零；而当输入的信号数值高过零时，它就会输出与输入一样的值。Leaky ReLU 和 ELU 函数则是 ReLU 变形修改而来的。

使用 MLP 处理点云数据[15, 18]时，将 N×D 的点云数据作为输入，设置特定的层数和节点的数目，那么就可以根据权重、偏置等参数来映射获得所需要的输出维度，N×C 的输出在一定程度上就反映出该点云在特征空间的分布，如此便将点云从几何空间投影到了特征空间。如图 3-1 所示，为 N×3 点云进行 3 输入-8 输出的一层 MLP 处理示意图。而对于运算参数的最佳选取，就是要借助网络学习来解决最优化问题，并在训练过程中持续地进行参数的迭代和更新，直到满足某个条件为止。

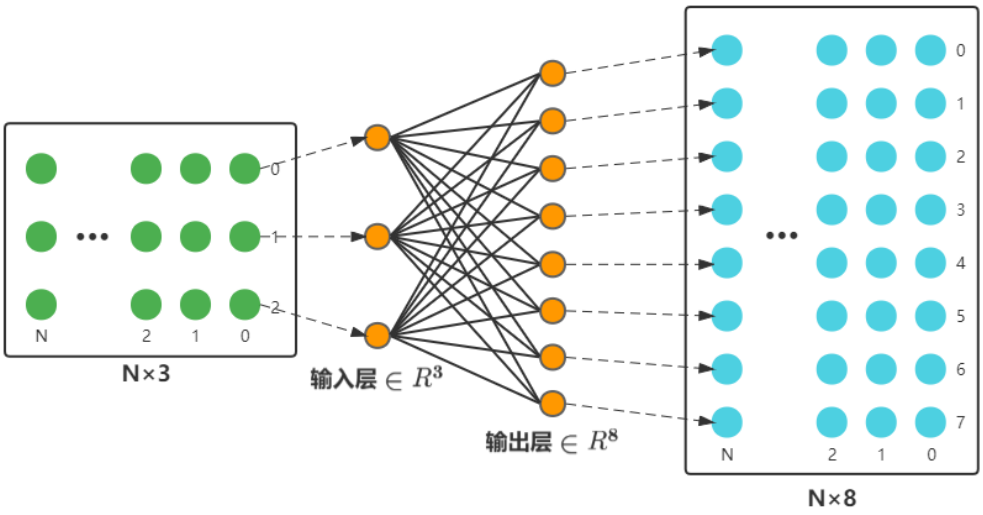


图 3-1　点云的 MLP 处理示意图

3.2.3　对称函数处理

点云格式的数据是无次序、不规则的，它不可以直接运用普通的二维平面图像卷积。如图 3-2 所示，我们假设两个大小都为 5×C 的点云 A、B，其中 C 代表点云的特征维度，

且它们表征的是同一个物体，只是点的排列顺序不同。在(2)和(3)式中，借助$K = [k_1\ k_2\ k_3\ k_4\ k_5]^T$的 5×C 固定卷积核，对 A 和 B 两个点云进行神经网络运算$g(\cdot)$获得点云特征f_A和f_B。为便于直观计算，这里的$g(\cdot)$可以简化为逐点的乘积和运算。

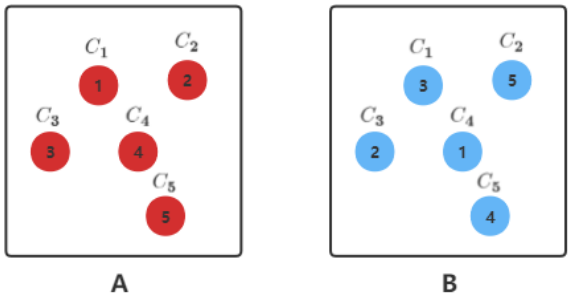


图 3-2　对点云无序性的运算说明

$$f_A = g\,(K, [\,C_1\ C_2\ C_3\ C_4\ C_5\,]^T) \tag{2}$$

$$f_B = g\,(K, [\,C_4\ C_3\ C_1\ C_5\ C_2\,]^T) \tag{3}$$

可以发现，由于点的顺序不同，直接应用神经网络计算，大部分情况下$f_A \neq f_B$，说明点云 A 和 B 的特征是不同的。然而，两者只是改变了点的顺序，它们仍然是相同的点云，这说明了直接进行卷积的结果中会去掉形状特征[19]，而遗留下无意义的点顺序信息。

Qi 等人[15]指出有三种方式可以使点云排列保持一致：对点进行指定的排序；以随机顺序的方式将点云送进循环神经网络 (Recurrent Neural Network, RNN) 中进行训练；利用对称函数处理为一个顺序不变的新向量。对于高维空间排序往往是不稳定的，网络也很难学习到一致的映射，而 RNN 处理大规模的点云序列性能也不佳。PointNet 创造性地使用对称函数$h(\cdot)$，使得输出的数值不会随着输入变量之间的排列次序发生变化，常见的对称函数比如求和($\sum$)、求积($\prod$)、最大(max)、最小(min)等。

$$f(\{x_1, x_2, x_3, \dots, x_n\}) \approx h\big(g(x_1), g(x_2), g(x_3), \dots, g(x_n)\big) \tag{4}$$

最常见的则是选择使用最大池化操作作为对称函数$h(\cdot)$，将神经网络运算$g(\cdot)$学习到的高维特征向量最大池化，从而聚合得到无序点云不变的属性，并在学习中不断接近真实的点云特征。

3.3　方案设计

3.3.1　总体架构

如图 3-3 所示，先使用一个深度神经网络提取输入点云的特征，重新生成优化分布后的点云，再由网络得到的特征与法向信息进行曲率计算，图(b)则是对曲率值进行的可视化，颜色越亮，曲率越大，说明该局部区域特征明显。最后根据曲率大小选择更少的点数，得

到最终的重采样结果。

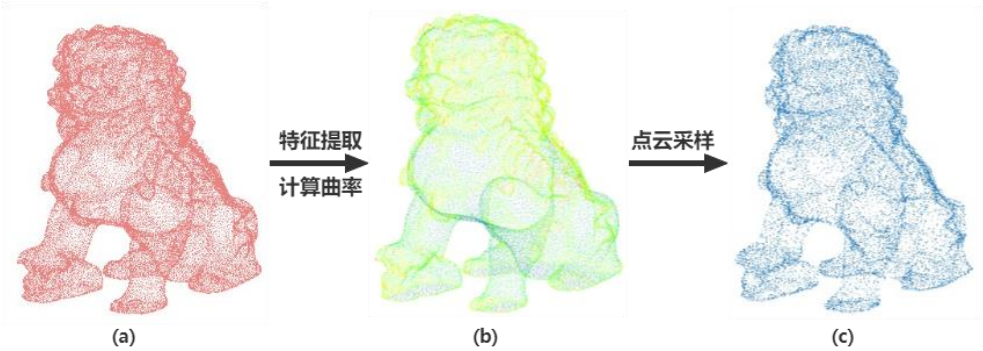


图 3-3　采样总体流程

如图 3-4 所示，为所设计的重采样架构图。对于 N×3 的点云，先使用设计的多级跃层连接模块 (Cascaded Skipping Connection Module, CSCM) 进行深度学习，提取点云特征，其维度为 N×264。这里的连接思想来源于 Huang 等人[36]在处理二维图像时所提出的 DenseNet。使用 4 个 CSCM 模块依次连接进行捕获点云信息，在模块内部通过一系列的跳跃连接、复制和级联拼接操作，以及对每次 CSCM 模块的输出，都串联上之前模块的输入后再进入下一个模块。这样既考虑到点之间的局部关系，又能学习到全局特征，具体的设计在后续的 3.3.2 节中解释。

对于特征空间不同维度的处理，我们仍然采用多层感知机的方式[15]。在 264 维特征矩阵从最后一个 CSCM 输出后，通过两个相同的多层感知机 MLP，分别得到新的点云坐标和每个点的法向信息。这里设置多层感知机 MLP 的维度依次为 128、128、64、3。获得的新点云就蕴含了我们根据神经网络提取到的特征并据此优化点的分布，而法向信息也是由学习到的特征转化而来，并不再是基于原始点云的传统几何方法拟合结果。最后，根据空间中各点的位置坐标和法向信息求出曲率值，依照曲率进行适应性降采样，得到局部分布均匀、特征自适应的高保真点云。

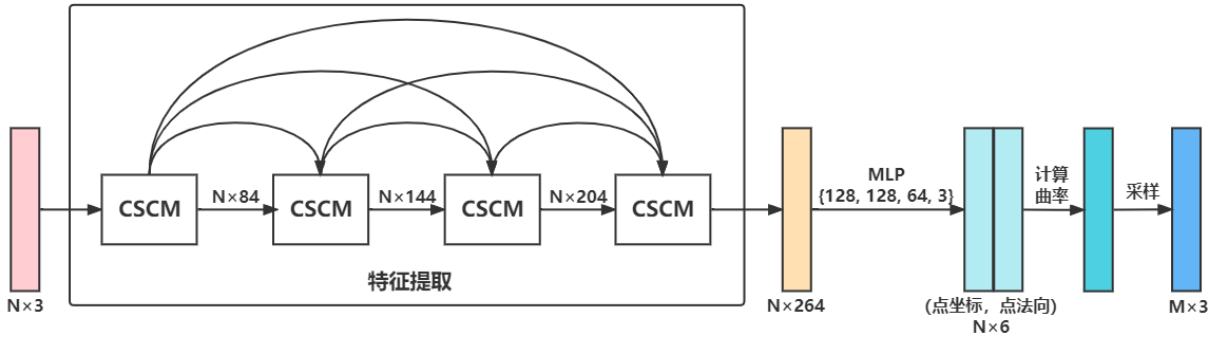


图 3-4　总体架构设计图

3.3.2 多级跃层连接模块

大多数现有的网络为了从输入点集 N×3 中提取结构特征 N×C，都是对输入点集进行分层处理来学习并提取出点云的多尺度特征矩阵，在网络结构中每一层前后直接顺次连接而成。而我们借鉴参考了包含密集连接操作的卷积网络 DenseNet[36]，在多个层次之间引入跳跃连接操作则可以跨层地利用提取的特征，高效加强了点云特征的传递，也在一定程度上减少了参数的数量。

多级跃层连接模块结构如图 3-5 所示，这里仅展示第一个 CSCM 模块。在每个 CSCM 中，首先将输入矩阵转换为固定数量的特征，使用特征 KNN (Feature KNN) [20]对特征进行分组，通过包含跳跃连接的一系列 MLP 细致地学习分组后的特征，然后选择最大池化 (Max Pooling) [15, 18]操作计算点的特征。

从细节上看，起初将输入点云通过一层的感知机扩展为 24 维的特征，这样便于该模块能够接收任意尺寸维度的数据。然后，受到 DGCNN[20]中 EdgeConv 模块的启发，我们在每个 CSCM 中都使用特征空间中的 K 邻近算法 (K-Nearest Neighbor, KNN)。我们设置 K=16，这样对于每个点，寻找它周围最邻近的 15 个特征，再和自己本身的特征一起构成一个大小为 N×16×24 的矩阵。可以发现，每一个点是在特征空间中进行最近邻的寻找，而每一次 CSCM 模块的特征感知都会改变特征空间中的值，这与在固定的坐标空间中执行 KNN 是不同的。因此，这个邻域是动态变化的，邻域所代表的局部信息也是不断更新的。

后续过程中，使用维度为 12 的 MLP 处理输出的特征信息，同时把最开始输入进 CSCM 的 N×24 矩阵复制 16 次，从而便于将两者拼接起来形成 N×16×36 的矩阵，强化了特征信息的前后传递。接着，连续使用 12 维的 MLP 和一个跳跃连接操作，不断将上层信息级联到下层。如此重复之后，通过最大池化处理得到 N×60 的输出特征。为了便于说明设计的细节，图 3-5 中的第一个 CSCM 模块包含了将该层的输入串联到下层输入的密集连接操作，即得到 N×84 的特征矩阵。故这里的结构图只需要首尾相连 4 个 CSCM 模块即可，处理过程依次类推，最后得到 N×264 的输出特征。

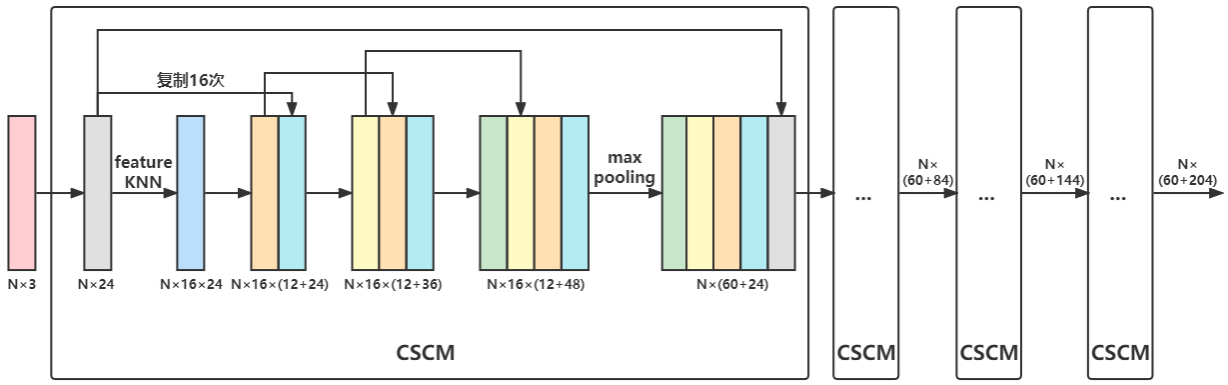


图 3-5 多级跃层连接模块结构图

3.3.3 基于曲率的采样

经典的传统采样方法包括随机采样 (Random Sampling, RS)、最远点采样 (Farthest Point Sampling, FPS)[37]和泊松圆盘采样 (Poisson Disk Sampling, PDS)[38]，其中 FPS 和 PDS 都是以迭代的方式生成样本。从一个随机样本开始，最远点采样 FPS 迭代地选取距离已经采样的点最远的点，它旨在实现输入点集的最大覆盖。FPS 易于实现，并且在许多处理分类任务的神经网络中广泛采用。泊松圆盘采样 PDS 通过设置两个点之间的最近距离，迭代地在该距离的圆环上采样，从而确保任意两点之间不超过这个指定距离，它产生比 FPS 更均匀的分布，但在曲面上生成泊松圆盘样本时计算成本很高。这些都只考虑了整体的均匀性，并未关注点云的特征分布。因此，我们可以使用反映点云特征的曲率来采样，保证点云的特征保持。

曲率是对平面中曲线的弯曲程度的度量，而对于一个三维点云，其各点的平均曲率代表了曲面嵌进三维空间内的弯曲状态，高斯曲率则表征曲面的凹凸性质。对于点云中的任何一个点，都有一个曲面可以无限接近于该点的邻近点云，对这个点和它的邻近点云进行曲面拟合，那就可以获得该点的曲率。但是，精准的曲率往往需要精准的法向进行计算，这也是大多传统方法[7-9]所面临的棘手问题。我们利用深度学习点云特征后，就可以输出优化后的点云和较为准确的法线信息，再估算所有点的曲率，从而可以根据曲率不断在局部空间中挑选出具有代表性的点，尽量充分地保持局部特性。如图 3-6 所示，为均匀的采样与特征保持的采样对比效果。在曲率值变化程度较大的局部领域，点云所描述的表面曲面变化也较大，说明该地方并不光滑，有显著并且丰富的特征，我们在此选取更多的点；反之，该局部区域是光滑程度较高的面，我们选取更少的点，最终的点云能够确保局部的均匀性和整体的特征自适应性。

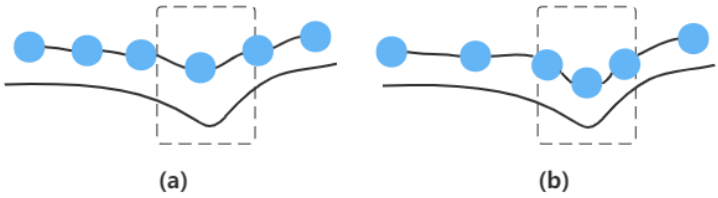


图 3-6 均匀的采样与特征保持的采样对比

第四章 技术具体实现

4.1 网络结构的关键实现

使用 Python 开发语言，借助拥有强大张量运算与深度神经网络框架的 PyTorch 机器学习库，实现了点云重采样技术中的特征提取。

4.1.1 多级跃层连接模块实现

多级跃层连接模块的类 CSConnectionModule 中关于特征 KNN 的调用和 forward 方法重写的关键部分如下。

```
class CSConnectionModule(nn.Module):
    def get_local_feature(self, x, k, id_point=None): # 特征 KNN
        # 输入: x: (B, C, N), id_point: (B, N, k)
        # 输出: knn_feature: (B, C, N, k)
        if id_point is None:
            knn_point, id_point, _ = group_knn(k + 1, x, x, unique=True)
            id_point = id_point[:, :, 1:]
            knn_point = knn_point[:, :, :, 1:]
        knn_center = torch.unsqueeze(x, dim=-1)
        knn_center = knn_center.expand_as(knn_point)
        knn_feature = torch.cat(
            [knn_center, knn_point - knn_center], dim=1)
        return knn_feature, id_point

    def forward(self, x, id_point=None):
        for i, mlp in enumerate(self.mlps):
            if i == 0: # 模块开始的连接处理
                y, id_point = self.get_local_feature(x, k=self.k, id_point=id_point)
                x = x.unsqueeze(-1).repeat(1, 1, 1, self.k)
                y = torch.cat([nn.functional.relu_(mlp(y)), x], dim=1)
            elif i == (self.n - 1): # 模块末尾的连接处理
                y = torch.cat([mlp(y), y], dim=1)
            else: # 模块中间的连接处理
                y = torch.cat([nn.functional.relu_(mlp(y)), y], dim=1)
        y, _ = torch.max(y, dim=-1) # 最大池化处理
```

```
        return y, id_point
```

4.1.2 特征提取实现

特征提取部分的类 FeatureExtraction 关键实现如下所示。

```
class FeatureExtraction(torch.nn.Module):
    def __init__(self, connect_num=3, fixed_dim=12, knn=16): #  特征提取类的构造函数
        super(FeatureExtraction, self).__init__()
        self.connect_num = connect_num

        input_dim = 3
        self.unit0 = Conv2d(3, 24, [1, 1], activation=None)
        self.unit1 = CSConnectionModule(
            24, fixed_dim=fixed_dim, n=connect_num, k=knn) #  第 1 个 CSCM 模块
        input_dim = 84
        self.pre_unit2 = Conv1d(input_dim, 24, 1, activation="relu")
        self.unit2 = CSConnectionModule(
            24, fixed_dim=fixed_dim, n=connect_num, k=knn) #  第 2 个 CSCM 模块
        input_dim = 144
        self.pre_unit3 = Conv1d(input_dim, 24, 1, activation="relu")
        self.unit3 = CSConnectionModule(
            24, fixed_dim=fixed_dim, n=connect_num, k=knn) #  第 3 个 CSCM 模块
        input_dim = 204
        self.pre_unit4 = Conv1d(input_dim, 24, 1, activation="relu")
        self.unit4 = CSConnectionModule(
            24, fixed_dim=fixed_dim, n=connect_num, k=knn) #  第 4 个 CSCM 模块
```

4.2 数据集

ModelNet40 数据集[17]是一个用于计算机三维图像与视觉研究的大规模立体 CAD 模型集合。该数据集中的类型有 40 个，总共 12311 个三角形网格组合而成的物体模型。由于设计之初是用于分类的目的，每个物体都有对应的类别标签，每一个类别里面也划分出了训练和测试两个数据集。如图 4-1 所示，数据集中每种物品的差异很大，为了便于使用，我们仅选择 10 个类别的部分物体作为我们的数据集。

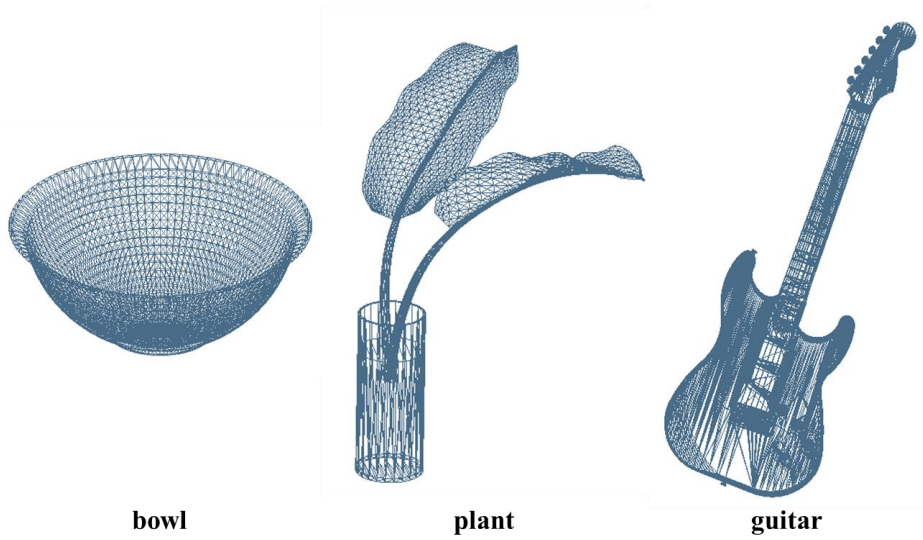


图 4-1 数据集 ModelNet40 中的部分物体

由于 ModelNet40 都是网格模型，使用 OFF 文件格式存储每个物体的点和三角形小面片信息，我们对于所使用到的物体进行随机采样大量的点，且使用点的位置坐标信息进行 PCA 法向估计[10]，从而获得一定数目的点云。测试集和训练集仍然依据原本的 ModelNet40 划分，最后我们使用的训练数据集包含 3991 个，所对应的测试数据集包含 908 个。另外，我们也选取了一些特征更复杂的扫描数据集作为测试和可视化。

4.3 损失函数

在训练深度神经网络的过程中，损失函数应该不断促进生成的点集更真实、更高质量地分布在物体表面上。因此，设计了一个综合重建损失L_{rec}、排斥损失L_{rep}和法向损失L_{nor}的联合损失函数。

对于重建损失，使用两个点云之间的倒角距离 (Chamfer Distance, CD)[39]，并进行部分加权设置。它由两部分组成，其中P为原始的输入点云，Q为经过网络学习后的输出点云。

$$L_{c1}\left(Q,P\right)=\frac{1}{|Q|}\sum_{q\in Q}\min_{p\in P}\|q-p\|^2 \tag{5}$$

$$L_{c2}\left(Q,P\right)=\frac{1}{|P|}\sum_{p\in P}\min_{q\in Q}\|p-q\|^2 \tag{6}$$

L_{c1}保证生成点云与输入的原始点云之间的距离最小，使得Q中的点尽可能接近P中的点。L_{c2}则保证原始点云在生成点云上的覆盖率，使得生成的点能很好地分布在输入点集上。将这两项加权后求和，即为重建损失L_{rec}。

$$L_{rec}\left(Q,P\right)=L_{c1}\left(Q,P\right)+\alpha L_{c2}\left(Q,P\right) \tag{7}$$

对于排斥损失，主要针对输出点云的局部邻近区域[29, 40]，如下所示。

$$L_{rep}\left(Q\right)=\frac{1}{m\cdot \mathrm{k}}\sum_{i=1}^{m}\sum_{j\in K(i)}\eta\left\|q_j-q_i\right\| \tag{8}$$

其中，m为输出点云Q的点数，$K(i)$为对每一个点q_i寻找k邻近点序列的索引。我们选择$\mathrm{k}=6$，以及使用的排斥项为$\eta(r)=-r$。排斥损失保证了生成的点在局部 KNN 领域上能分散开来，不会过于靠近而导致聚集在一起。

对于法向损失，如下所示。

$$L_{nor}\left(Q,P\right)=\frac{1}{m}\sum_{i=1}^{m}\left\|n_i-N\left(p_i\right)\right\|^2 \tag{9}$$

其中，m仍为生成点云Q的点数，$N\left(\cdot\right)$则表示对点使用传统的 PCA 法向估计[10]获得的法向坐标。

联合上述损失，即构成了我们的损失函数L_{net}。

$$L_{net}\left(Q,P\right)=L_{rec}\left(Q,P\right)+\beta L_{rep}\left(Q\right)+\gamma L_{nor}\left(Q,P\right) \tag{10}$$

根据实验研究发现，我们设置的权重为$\alpha=0.01$以及$\beta=0.006$。由于网络在进行重建点云时已经保证了点的相似性，点云特征中的法向信息也会有所学习，故该法向损失的权重设置可以较小，为$\gamma=0.001$。

第五章 实验结果

5.1 对比其他方法

5.1.1 评价指标

为了将所设计的重采样方法与其他方法进行定量比较，我们引入两个点云之间近似性的评价指标。最常用的评价方式包括均值和均方根 (Root Mean Square, RMS)[41]、归一化均匀系数 (Normalized Uniformity Coefficient, NUC)[29]、倒角距离 (Chamfer Distance, CD)[39]、点到面的距离 (Point-to-Surface, P2F)[42]、搬土距离 (Earth Mover's Distance, EMD)[43] 和豪斯多夫距离 (Hausdorff Distance, HD)[44] 等。

对于两个点云P和Q，它们之间的倒角距离以及豪斯多夫距离如下所示。其中，p和q分别代表点云P和Q中的一个点。倒角距离主要用于评估两个点集之间最短点距离的平均值，而 HD 则使用距离的最大最小函数，衡量两个点集的最大不匹配程度。

$$\mathrm{CD}\ (Q,P) = \frac{1}{|Q|}\sum_{q\in Q}\min_{p\in P}\|q-p\|^2 + \frac{1}{|P|}\sum_{p\in P}\min_{q\in Q}\|p-q\|^2 \tag{11}$$

$$\mathrm{HD}\ (Q,P) = \max\left(\max_{q\in Q}\min_{p\in P}\|q-p\|^2, \max_{p\in P}\min_{q\in Q}\|p-q\|^2\right) \tag{12}$$

在衡量输出采样点云与真实网格模型之间的相似程度时，我们使用点到面距离 P2F 的平均值来说明两者的质量差异。

5.1.2 定量比较

实验的比较对象包括随机采样 (RS)、最远点采样 (FPS)[37]、泊松圆盘采样 (PDS)[38]，以及 Self-sampling[45]方法，这些采样都是与下游任务无关的，强调关注点云本身特点，且无需额外的标签。Self-sampling 是关于点云整合的神经网络，它反复使用全局点集的子集对输入点云进行自采样，网络不断学习利用单个子集形状中固有的特性，最终调整重构出一个新的点集，该点集能重新刻画出物体表面形状。

表 5-1 在测试数据集上对不同方法重采样结果的定量比较。评价指标 CD、HD、P2F 数值越小，说明采样越接近原始点云

采样方法	CD (e^{-3})	HD (e^{-3})	P2F (e^{-3})
RS	18.02	47.38	2.92
FPS	12.74	39.97	2.15
PDS	14.02	20.23	2.43
Self-sampling	16.45	10.82	1.78
本文方法	**8.71**	**10.69**	**1.73**

用不同的方法对点云的测试数据集进行了重采样实验，每个物体输入的点数都为 2048，

定义采样率为 $Sampling\ Ratio = N/M = 2048/M$。当点云的输出点数是 1024，即设置的采样率等于 2 时，各种方法的 CD、HD、P2F 数值如表 5-1 所示。虽然 Self-sampling 也是基于学习的采样，但在此采样率下，我们的 CD 值更小，点云之间的平均距离更小，我们的方法在形状保真度上取得了更好效果。

在不同采样率的取值下，输出点云的大小为$k \in \{2, 4, 8, \dots, 1024\}$，计算使用不同方法后结果的倒角距离，如图 5-1 所示。从中可以看到，RS 表现最差，它的随机性无法捕获有意义的信息。与 PDS 相比，FPS 由于尽可能选取偏离最远的点，在降采样比例更大时更有可能选取边、角等边缘点，略微提升采样效果。我们的方法与 Self-sampling 十分接近，不同采样率下都能保持较低的 CD 值，不会过于受到点云输出大小的影响，并且我们采样后的点云与原始输入相似程度更佳。

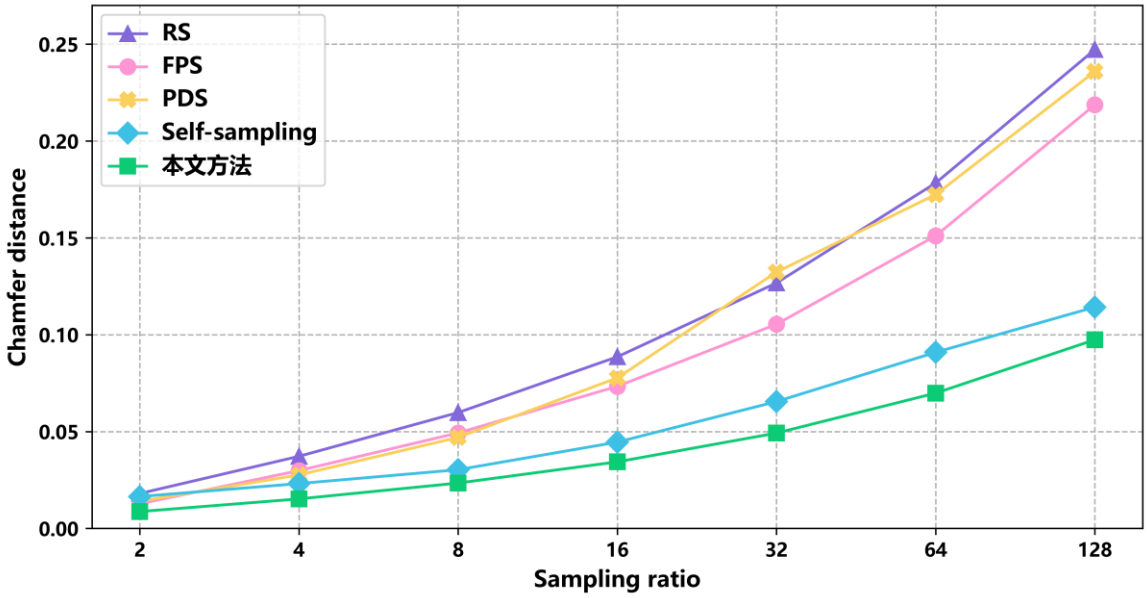


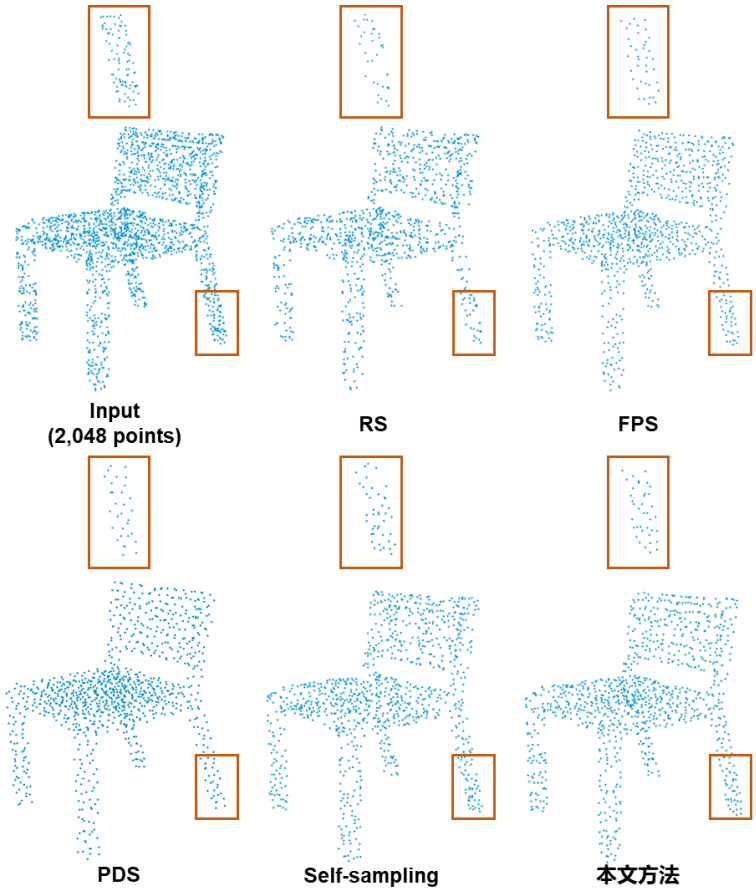
图 5-1 不同采样率下的比较。输入点云均为 2048 个点，使用不同的采样方法对测试集点云采样，并进行 CD 评估

5.1.3 定性比较

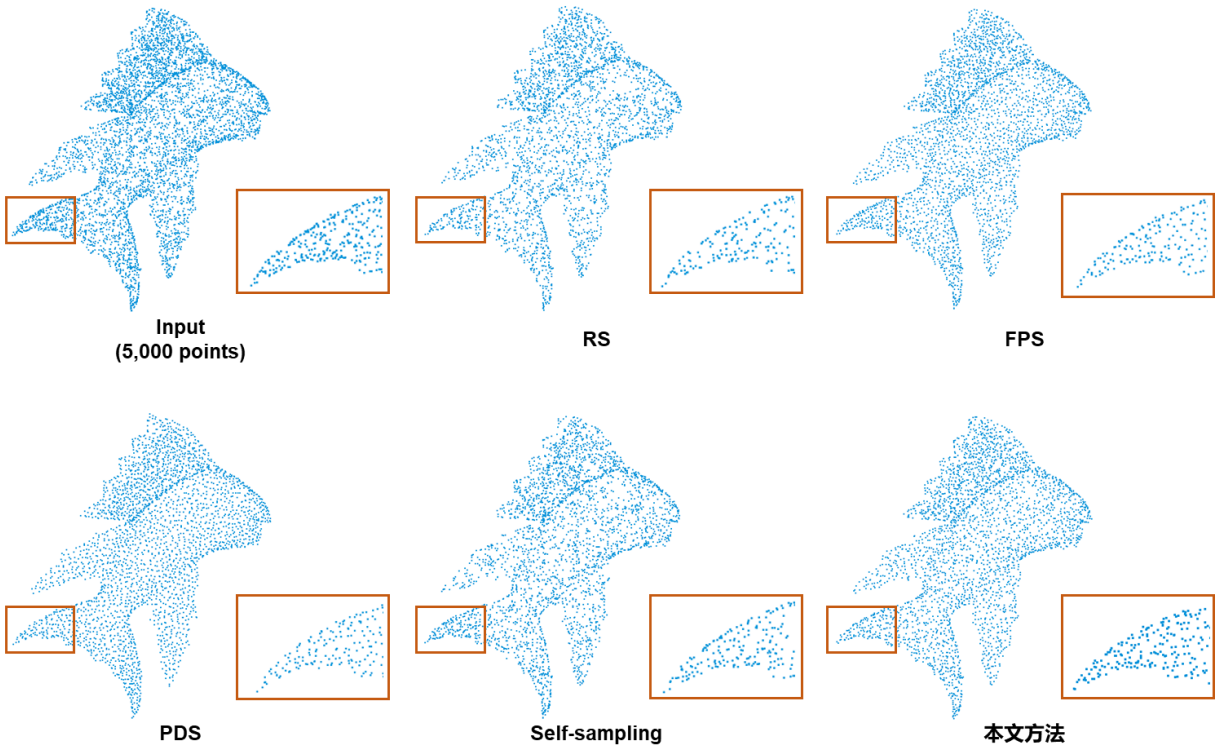
如图 5-2 所示，在与其他方法定性比较时，我们选取了测试集中的 Chair 模型（2048 个点）以及两个更复杂的扫描数据模型 Fish（5000 个点）与 Chinese Lion（50000 个点），分别降采样至 1024、3000、10000 的点数。

可以看到，RS 采样点云过于杂乱，点集的分布完全没有均匀性，而 FPS 和 PDS 由于算法本身的特点，保持了良好的全局均匀性，尤其是在大规模点集上更为明显（例如 Chinese Lion 点集）。Self-sampling[45]可以重新整合点云，优化强调边缘周围的点，产生的结果有明显的特征轮廓，然而这一方法在重新生成点时往往会产生异常和缺陷（例如 Chair 点集的腿部分），分布均匀性也不够好（例如 Fish 点集的鱼尾部分）。我们所使用的方法能够在局部尽量保持均匀性的前提下，更为真实地保留物体的特征区域，也不会引入其他的异常点，

整体上的点云保真度也更高。



(a)



(b)

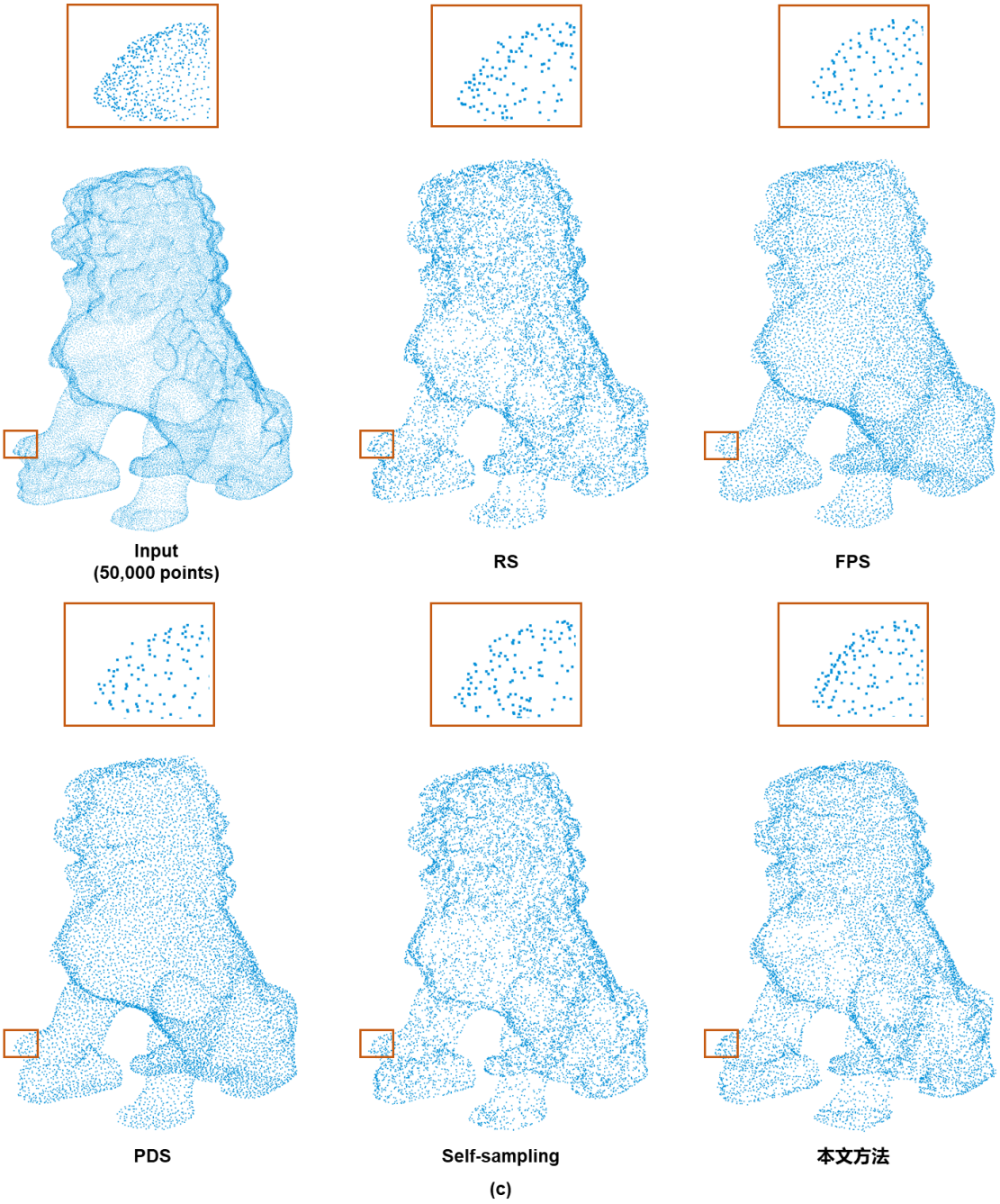


图 5-2 不同方法重采样结果的定性比较。对于每个模型，显示输入点集和不同方法下的输出点集，同时使用放大视图显示详细的点集分布。其中，(a)为 Chair 点云模型，(b)为 Fish 点云模型，(c)为 Chinese Lion 点云模型

5.2 三维重建

我们对重采样后的实验结果进行三维重建，采用经典的泊松表面重建[32]方法，而重建的误差由 CD 值[39]进行衡量。为了便于统一比较不同的采样方法，借助归一化重建误差(Normalized Reconstruction Error, NRE)[29] 这一指标进行计算，即分别将降采样点云和输入

点云重建后，取两者之间的重建误差比。NRE 的值最小为 1，值越小，说明重建效果越好。

$$NRE\ (Q,P) = \frac{CD\ (Q_{rec},P)}{CD\ (P_{rec},P)} \tag{13}$$

其中Q_{rec}、P_{rec}分别为点云Q、P重建后的新点云。

我们仍然定义采样率为$Sampling\ Ratio = N/M = 2048/M$，在不同的采样率下，实验输出点云的大小为$k \in \{32, 64, 128, \dots, 1024\}$，比较在测试数据集上的平均 NRE，如图 5-3 所示。当采样率低时，所有的方法都具有相似的重建性能。然而，对于较高的采样率，我们的方法 NRE 值更低，展现出了一定的优势，说明其学习到了有意义的特征信息，选择了更有利于三维重建的点。但是，我们的方法在高采样率下的稳定性不及 Self-sampling[45]，如果对于超大规模点集，采样率极高时，重建误差猜测会进一步加大。这可能是我们在网络学习后根据曲率采样时，由于重采样点数相对较少，丢失了一些曲率并不大的特征点，影响了点云的重建。

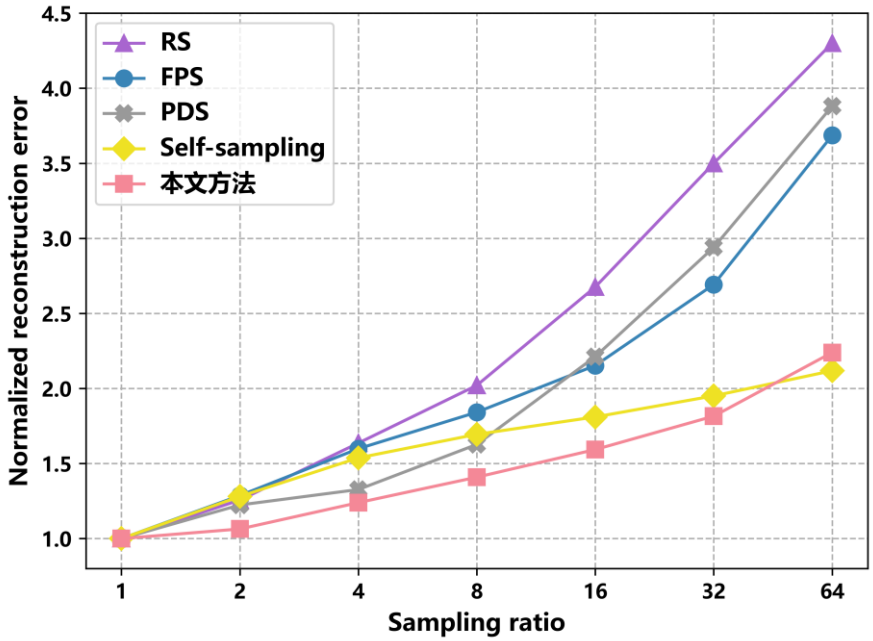


图 5-3 不同采样率下的三维重建结果比较。输入点云均为 2048 个点，使用不同的采样方法对测试集点云采样后重建，并进行 NRE 评估

选取数据集中的 Desk 模型进行可视化，可以直观地比较不同重采样方法所获得的点云重建后的网格模型，如图 5-4 所示。RS 采样后重建出的整体效果较差，有大量的异常表面，而 FPS 与 PDS 能明显看到点较为均匀地分布在表面上，但是在相同放大比例下，点数更多，反映出该重建区域越不平滑。而 Self-sampling 重建出的结果边缘突出，特征明显，但点的分布容易受到轮廓的影响导致不均匀。综合比较之下，我们的方法在重建结果上比其他方法质量更好，提高了局部重建表面的均匀性与平滑性，更接近原始的三维模型，也说明了我们能够很好地提取点云的显著特征并进行重采样。

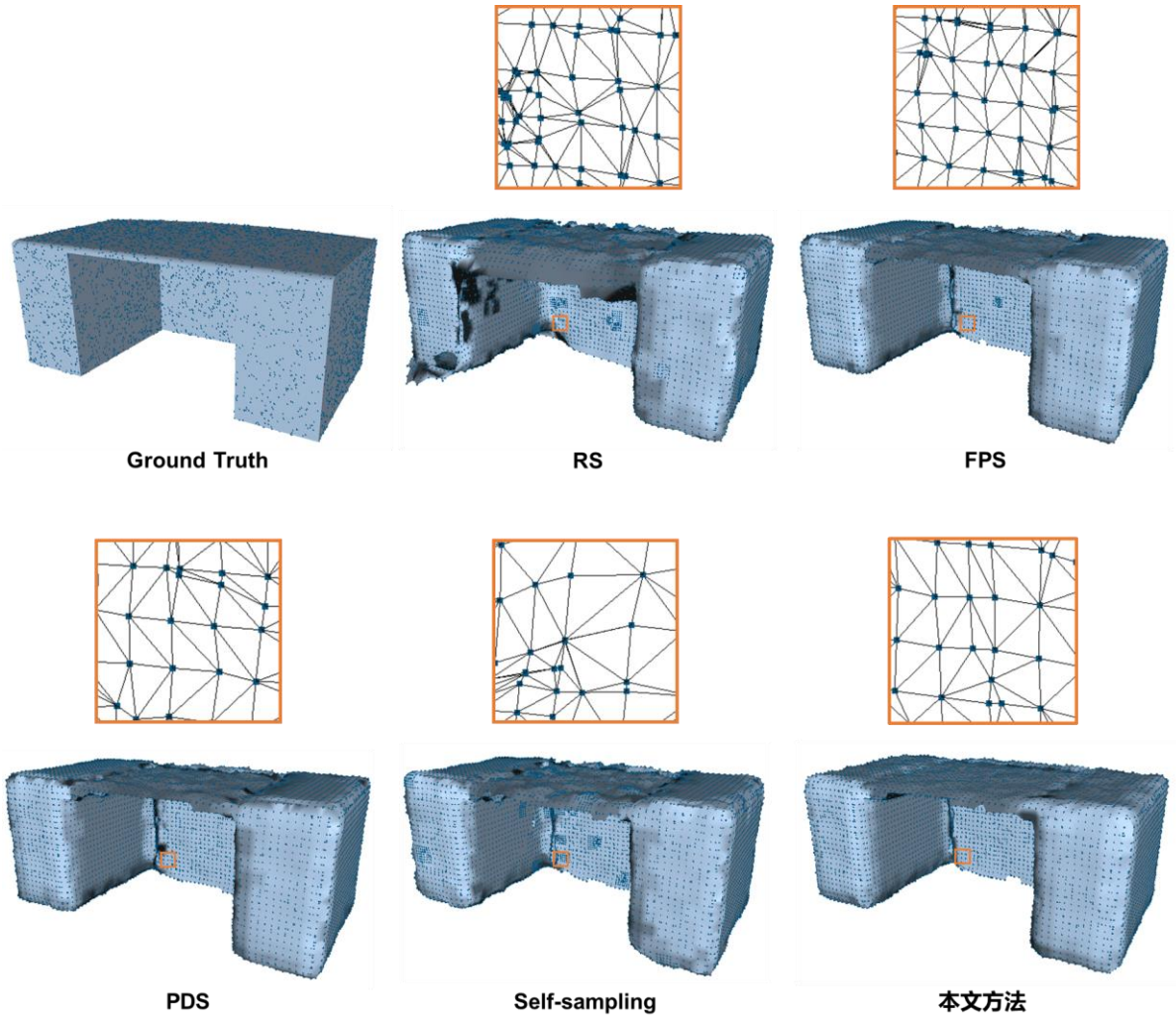


图 5-4 不同方法下三维重建结果的可视化。我们使用 Desk 模型，对输入点集（10000 个点）用不同方法采样 5000 个点输出并重建，同时使用放大视图显示详细的三角面片组成。注意 Ground Truth 为真实的网格模型

5.3 消融实验

将我们网络中的部分模块去掉，以评估它们在采样流程中所起到的有效性。如表 5-2 所示，测试集的输入点云大小均是 2048 个点，输出点云为 1024 个点，使用 CD[39]和 HD[44]评估了消融实验的结果。其中，去掉多级跃层连接部分是在每一个模块中直接使用特征 KNN 与最大池化处理特征空间。这和 DGCNN[20]中的 EdgeConv 处理基本相同，并且我们仍然连续使用四次。而去掉法向学习部分则是不再由网络输出法向坐标，我们用传统的 PCA 法向估计[10]方法代替。

实验结果说明，我们所设计的多级跃层连接模块确实能够加强对于点云特征的学习，多次 CSCM 模块的使用也能提高点云质量。如图 5-5，将网络输入点云与优化分布后的输出点云进行可视化，对比可以看到网络重新调整了点云，强调特征明显的区域与局部的均

匀性，更便于后续基于曲率的采样。另外，对于法向信息，传统估算方法过于依赖点云的几何分布，拟合的误差又会影响到点云的曲率和重建，所以直接学习获取法向在我们的网络中是很有必要的部分。

表 5-2 消融实验的结果。评价指标 CD、HD 数值越小，说明采样越接近原始点云

实验方法	CD (e^{-3})	HD (e^{-3})
去掉多级跃层连接部分	12.19	22.87
去掉法向学习部分	15.81	18.16
只使用一个 CSCM	9.04	10.80
完整流程	**8.71**	**10.69**

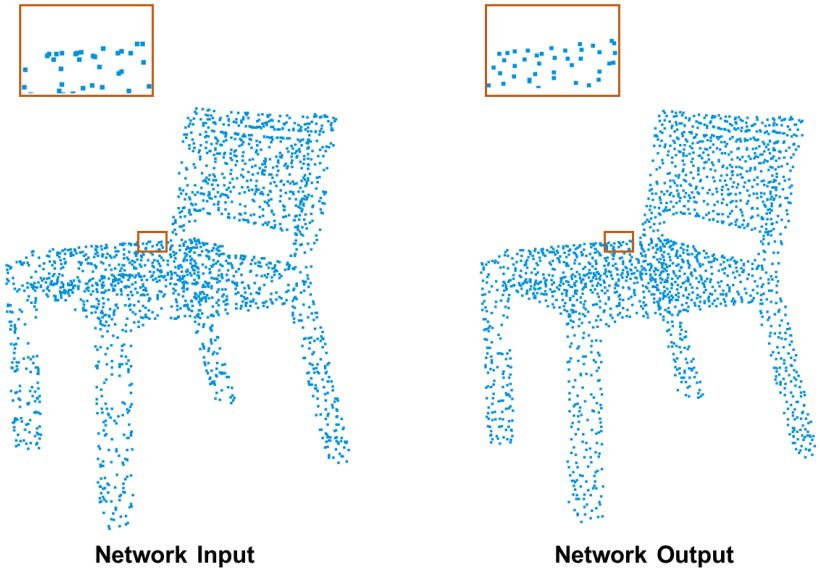


图 5-5 网络输入与输出比较。Chair 点云模型的网络输入输出前后均为 2048 个点，我们使用放大视图显示详细的点集分布。

5.4 参数的影响

在设计的网络损失函数中，存在α和β两个权重参数，前者是针对重建损失中的一项，后者是对排斥损失进行加权。如图 5-6 和 5-7 所示，我们研究并分析了不同参数下的网络对采样结果的影响。实验表明，采用合适大小的重建损失可以实现高保真度的点集选择，而排斥损失的引入又能促进点在局部空间分布的均匀性。我们使用$\alpha = 0.01$以及$\beta = 0.006$来达到尽可能好的结果。

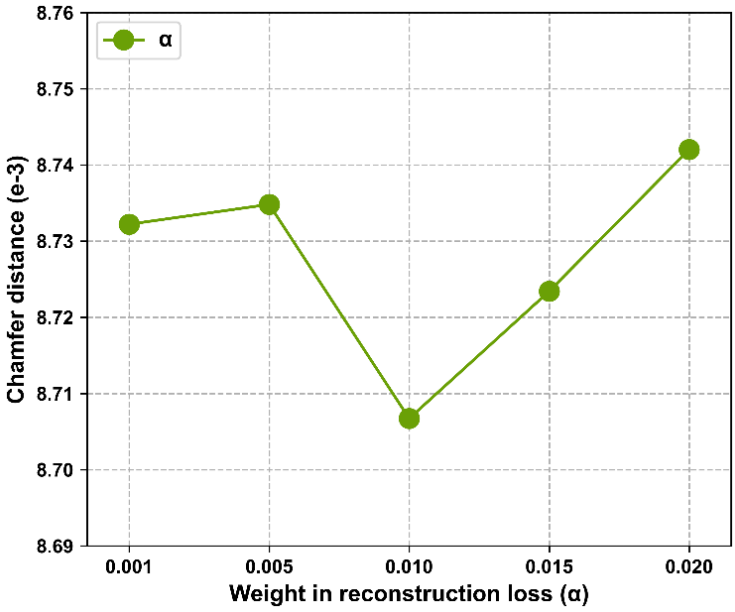


图 5-6 参数α的不同对采样的影响

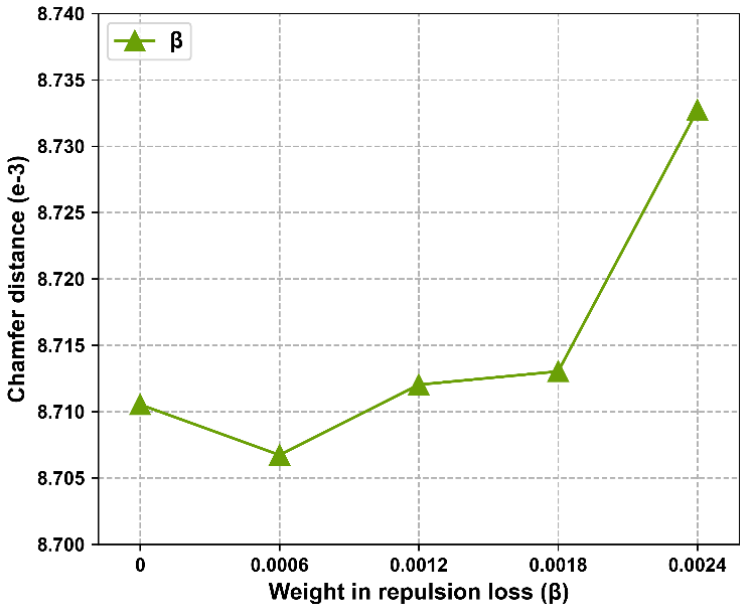


图 5-7 参数β的不同对采样的影响

5.5 健壮性分析

为了验证我们的采样方法的健壮性，添加不同幅度的噪声到输入点云中，计算并比较点云的 CD 值变化，如图 5-8 所示。我们采用常见的高斯噪声，不同幅度取值表示高斯噪声的标准差大小，只使用幅度为 4%的噪声输入作为网络训练，而在测试集上，分别使用的噪声输入幅度依次为 0、1%、2%、3%、4%。

实验结果表明，当含噪声的待采样点云输入时，我们的方法仍能够尽量提取接近原始

干净点云的特征。其中，FPS 的效果甚至比 RS 更差，这是因为它总是会选取最远的那些异常噪声点。与直接从点云中选择点的方法相比，基于学习的方法不会受到噪声的严重影响，即使输入点云的噪声幅度为 4%，仍然能保持较低的 CD 值。而 Self-sampling[45]方法在噪声输入后，也会对网络学习产生一定的干扰，我们的方法鲁棒性更强。

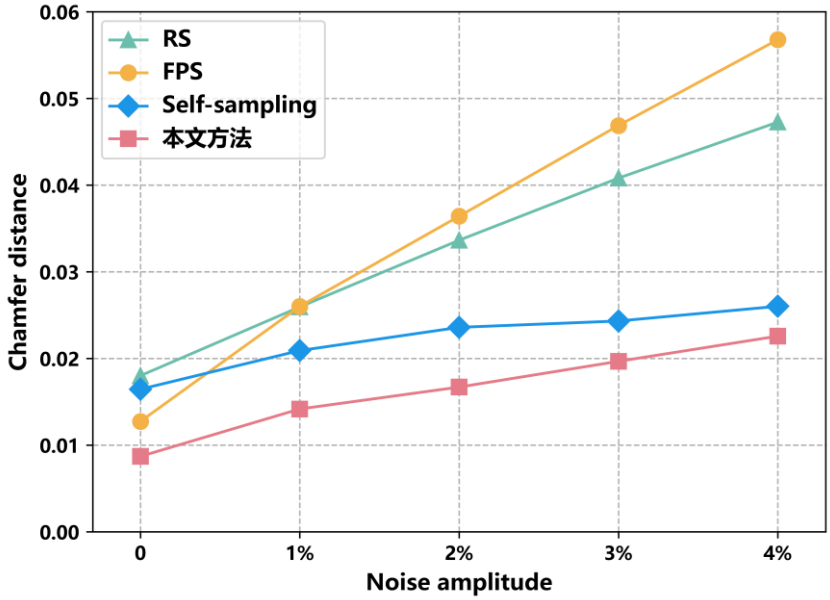
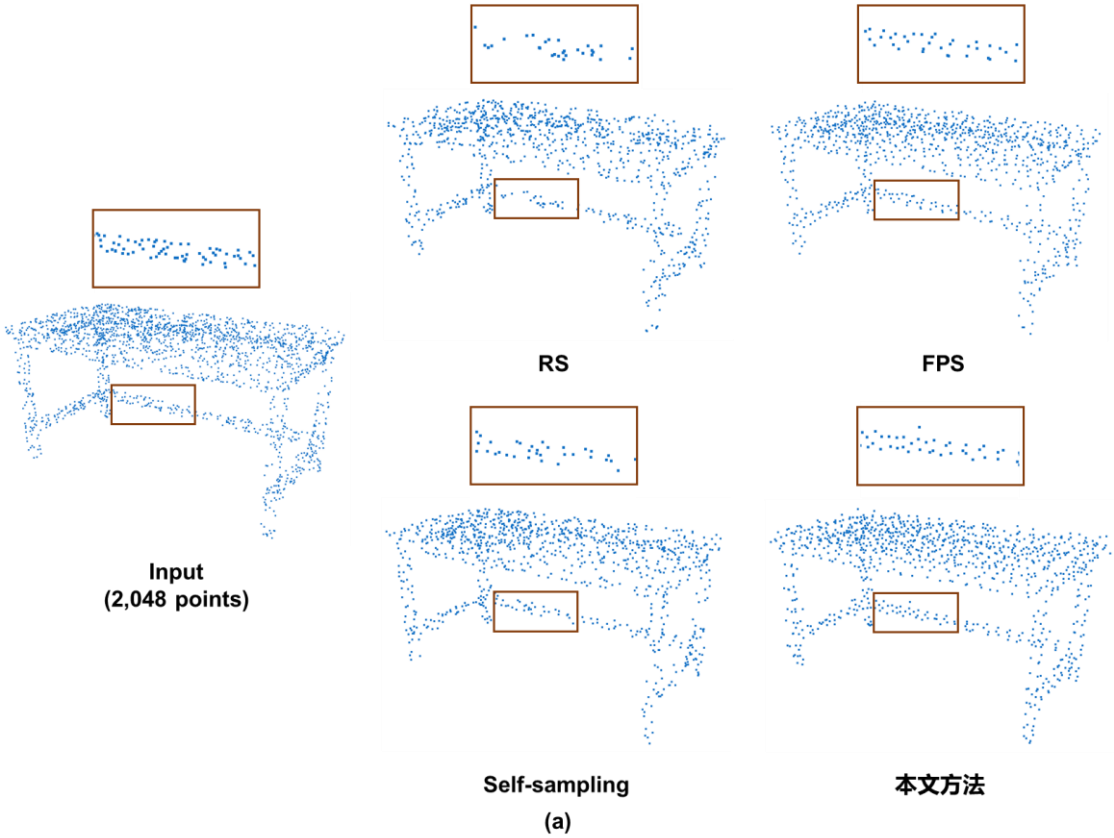


图 5-8 不同噪声幅度影响下的重采样结果比较。输入点云均为 2048 个点，使用不同的采样方法对测试集点云采样至 1024 个点，并进行 CD 评估



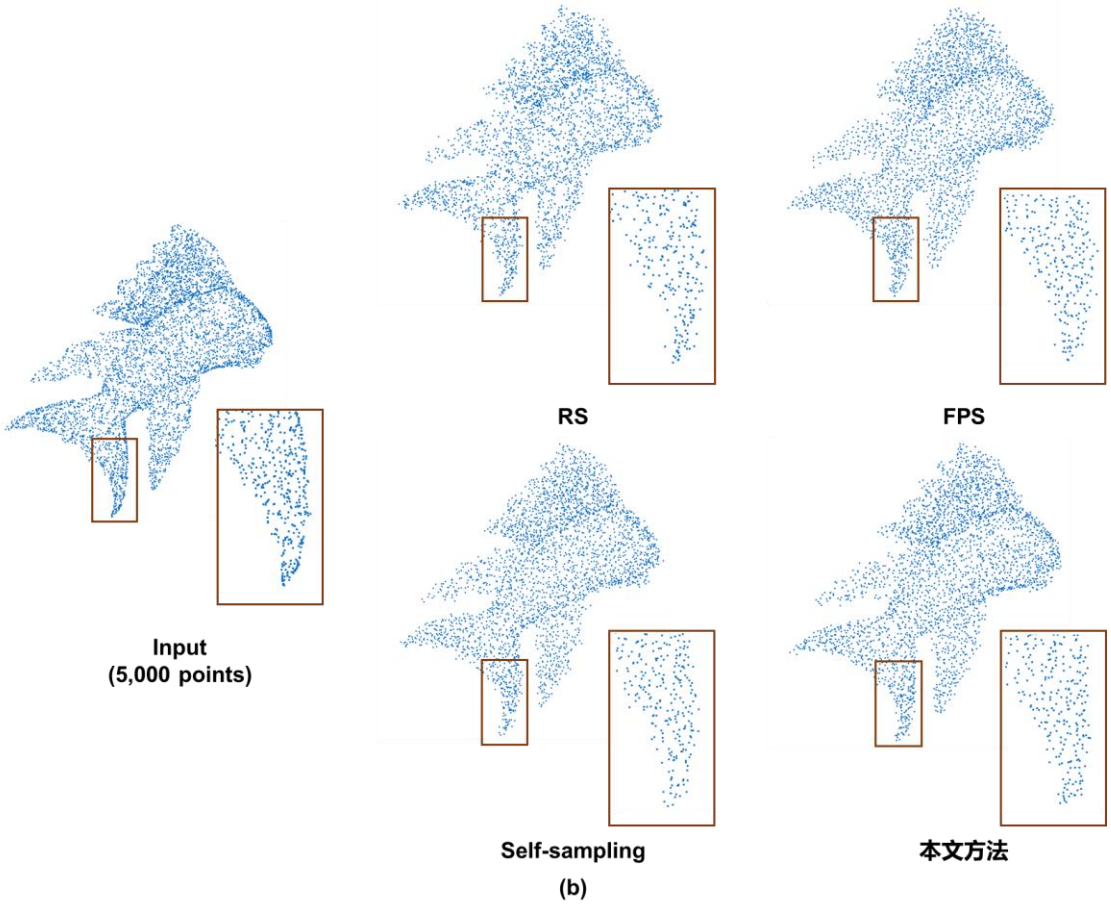


图 5-9 不同噪声幅度影响下重采样结果的可视化。其中，(a)是对 Table 输入点集（2048 个点）使用 2%的噪声幅度，并采样至 1024 个点，(b)是对 Fish 输入点集（5000 个点）使用 4%的噪声幅度，并采样至 3000 个点。使用放大视图以显示详细的点集分布

在图 5-9 中，展示了对输入点集添加噪声后重采样的结果可视化。可以看出，FPS 采样受到噪声的影响较大，选取了较多的异常点，输出的点集扩大了物体的轮廓。而 Self-sampling 在处理点云时，噪声的干扰一定程度上影响了对特征的判断，使得边缘特征不再明显。我们所采用的采样方法虽然也会出现异常点，但总体数量较少，在捕获特征区域时仍有优势，对噪声有较好的健壮性。

第六章 总结与展望

6.1 本文研究总结

近年来，有关三维点云数据的研究探索是计算机视觉与图形学中非常重要的课题之一。多种多样的图形建模需求和应用急剧增加，而高速发展的 3D 扫描设备又有助于随时获取复杂的现实世界对象。这些技术会产生大量信息丰富的点样本，却会由于各种外界环境和设备因素造成点云经常有丢失、冗余、噪声、不均匀等的特点。因此在三维重建等各种应用中，通常需要进行重采样点云的预处理工作。

本文总结了各种基于传统几何与基于深度学习方法的点云相关处理，尤其是对于深度学习相关的点云重采样研究，分析了它们的优劣之处，并针对目前的不足，设计了面向三维重建的点云重采样技术，通过实验说明了它显著的有效性。我们的方法表明，在引入深度学习的情况下，提取几何特征并将原始采样点云简化更有利于实现高质量三维重建。具体的研究工作如下：

（1）本文设计了一个多级跃层连接模块 (Cascaded Skipping Connection Module)，可以用于点云的特征提取。通过在特征空间上的特征 KNN 计算，以及各层感知机结果的跳跃连接、复制和级联操作，CSCM 充分利用了点云局部特征的同时，又兼顾考虑了点集的全局分布特点。这在后续重新生成新的点云分布与采样中具有重要的意义。

（2）本文构建了一个深度神经网络进行点云分布优化和法向信息生成，并根据点的位置坐标和法向进行基于曲率的降采样。训练基于 ModelNet40 中的部分数据集，并设计了一个包含重建损失、排斥损失和法向损失三项的联合损失，有效保证了结果尽量接近原始点云，以及局部区域的均匀性。

（3）本文实验验证了所设计的重采样技术能很好地生成点数更少、局部分布更均匀、特征自适应的点云。通过各种衡量数据指标以及可视化的结果显示，本文的方法在与原始输入点云的差异程度、点集分布的特征上均要优于其他对比方法，能够有效地实现高质量的三维重建。

6.2 未来研究展望

本文所设计的点云重采样技术相较于其他方法，取得了更好的实验成果。结合阅读的大量国内外研究文献以及点云数据在各个领域的创新应用，我们的方法仍存在改进的空间，主要如下：

（1）由于点云不同局部区域特征的差异性和复杂性，对于其特征的学习提取还不够充分。后续工作可以立足于几何处理的理论，借鉴显著性检测[46, 47]或者边缘轮廓检测[48, 49]等方法，强调每个点云的边、角等特征明显的区域对于输出的贡献，从而使网络能够细致精

准地学习不同的局部特征。

（2）当点云的规模过于庞大时，我们的方法在计算成本和存储空间上急剧扩大，效果的稳定性也会受到影响。而各种对点云进行分块分片的提取思想已被很多研究者所采用[50, 51]，如何处理分块点云并在全局上合并特征，提高大规模点云的处理效率是今后可以进一步优化的方向。

另外，虽然我们的研究是为点云降采样而设计的，但对于特征的提取思路也可以拓展到点云的上采样领域，用已有的特征去不断丰富缺失的信息。

总之，基于深度学习方向的三维点云相关研究正在蓬勃兴起，从最开始的简单分类到陆续开展的各种语义学习、特征提取等任务，人们越来越重视三维点云在各种各样的应用场景中所带来的强大效益。对于点云重采样，如何借鉴其他点云任务的研究方法，甚至将二维图形图像技术推广拓展，并根据实际情况改良和优化，更真实客观地反映现实世界的物体，都是这一领域未来的探索方向。

参考文献

[1] Yuke Z, Mottaghi R, Kolve E, et al. Target-driven visual navigation in indoor scenes using deep reinforcement learning[J]. 2017 IEEE International Conference on Robotics and Automation (ICRA), 2017: 3357-3364.

[2] Liang M, Yang B, Wang S L, et al. Deep Continuous Fusion for Multi-sensor 3D Object Detection[C]//Proceedings of the 15th European Conference on Computer Vision (ECCV), 2018, 11220: 663-678.

[3] Rusu R B, Marton Z C, Blodow N, et al. Towards 3D Point cloud based object maps for household environments[J]. Robotics and Autonomous Systems, 2008, 56(11): 927-941.

[4] Gross M. Getting to the Point...?[J]. IEEE Computer Graphics and Applications, 2006, 26(5): 96-99.

[5] Kobbelt L, Botsch M. A survey of point-based techniques in computer graphics[J]. Computers & Graphics-Uk, 2004, 28(6): 801-814.

[6] Alexa M, Behr J, Cohen-Or D, et al. Computing and rendering point set surfaces[J]. IEEE Transactions on Visualization and Computer Graphics, 2003, 9(1): 3-15.

[7] Lipman Y, Cohen-Or D, Levin D, et al. Parameterization-free projection for geometry reconstruction[J]. Acm Transactions on Graphics, 2007, 26(3).

[8] Huang H, Li D, Zhang H, et al. Consolidation of unorganized point clouds for surface reconstruction[J]. ACM Trans Graph, 2009, 28(5): 1–7.

[9] Liao B, Xiao C, Jin L, et al. Efficient feature-preserving local projection operator for geometry reconstruction[J]. Computer-Aided Design, 2013, 45(5): 861-874.

[10] Hoppe H, DeRose T, Duchamp T, et al. Surface reconstruction from unorganized points[C]//Proceedings of the 19th annual conference on Computer graphics and interactive techniques, 1992: 71–78.

[11] Huang H, Wu S, Gong M, et al. Edge-aware point set resampling[J]. ACM Trans Graph, 2013, 32(1): Article 9.

[12] Preiner R, Mattausch O, Arikan M, et al. Continuous Projection for Fast L-1 Reconstruction[J]. Acm Transactions on Graphics, 2014, 33(4).

[13] Öztireli A C, Alexa M, Gross M. Spectral sampling of manifolds[J]. ACM Trans Graph, 2010, 29(6): Article 168.

[14] Chen S, Tian D, Feng C, et al. Fast Resampling of Three-Dimensional Point Clouds via Graphs[J]. IEEE Transactions on Signal Processing, 2018, 66(3): 666-681.

[15] Charles R Q, Su H, Kaichun M, et al. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation[C]//Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition

(CVPR), 2017: 77-85.

[16] Su H, Maji S, Kalogerakis E, et al. Multi-view Convolutional Neural Networks for 3D Shape Recognition[C]//Proceedings of the 2015 IEEE International Conference on Computer Vision (ICCV), 2015: 945-953.

[17] Zhirong W, Song S, Khosla A, et al. 3D ShapeNets: A deep representation for volumetric shapes[C]//Proceedings of the 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015: 1912-1920.

[18] Qi C R, Yi L, Su H, et al. PointNet plus plus : Deep Hierarchical Feature Learning on Point Sets in a Metric Space[C]//Proceedings of the 31st Annual Conference on Neural Information Processing Systems (NIPS), 2017, 30.

[19] Li Y Y, Bu R, Sun M C, et al. PointCNN: Convolution On X -Transformed Points[C]//Proceedings of the 32nd Conference on Neural Information Processing Systems (NIPS), 2018, 31.

[20] Wang Y, Sun Y, Liu Z, et al. Dynamic Graph CNN for Learning on Point Clouds[J]. ACM Trans Graph, 2019, 38(5): Article 146.

[21] Kuang Z, Yu J, Fan J, et al. Deep Point Convolutional Approach for 3D Model Retrieval[C]//Proceedings of the 2018 IEEE International Conference on Multimedia and Expo (ICME), 2018: 1-6.

[22] Achlioptas P, Diamanti O, Mitliagkas I, et al. Learning Representations and Generative Models for 3D Point Clouds[C]//Proceedings of the 35th International Conference on Machine Learning (ICML), 2018, 80.

[23] Yew Z J, Lee G H. 3DFeat-Net: Weakly Supervised Local 3D Features for Point Cloud Registration[C]//Proceedings of the 15th European Conference on Computer Vision (ECCV), 2018, 11219: 630-646.

[24] Shi S, Wang X, Li H. PointRCNN: 3D Object Proposal Generation and Detection From Point Cloud[C]//Proceedings of the 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019: 770-779.

[25] Yang J, Zhang Q, Ni B, et al. Modeling Point Clouds With Self-Attention and Gumbel Subset Sampling[C]//Proceedings of the 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019: 3318-3327.

[26] Nezhadarya E, Taghavi E, Razani R, et al. Adaptive Hierarchical Down-Sampling for Point Cloud Classification[C]//Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020: 12953-12961.

[27] Dovrat O, Lang I, Avidan S. Learning to Sample[C]//Proceedings of the 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019: 2755-2764.

[28] Lang I, Manor A, Avidan S. SampleNet: Differentiable Point Cloud Sampling[C]//Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020: 7575-7585.

[29] Yu L, Li X, Fu C, et al. PU-Net: Point Cloud Upsampling Network[C]//Proceedings of the 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018: 2790-2799.

[30] Li R, Li X, Fu C, et al. PU-GAN: A Point Cloud Upsampling Adversarial Network[C]//Proceedings of the 2019 IEEE/CVF International Conference on Computer Vision (ICCV), 2019: 7202-7211.

[31] Qian Y, Hou J, Kwong S, et al. PUGeo-Net: A Geometry-Centric Network for 3D Point Cloud Upsampling[C]//Proceedings of the Lecture Notes in Computer Science: Springer International Publishing, 2020: 752-769.

[32] Kazhdan M, Bolitho M, Hoppe H. Poisson surface reconstruction[C]//Proceedings of the fourth Eurographics symposium on Geometry processing, 2006: 61–70.

[33] Kingma D P, Ba J L. Adam: A method for stochastic optimization[C]//Proceedings of the 3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings, 2015.

[34] Chaudhary A, Chouhan K S, Gajrani J, et al. Deep Learning With PyTorch[M]. Machine Learning and Deep Learning in Real-Time Applications, 2020.

[35] Zhou Q-Y, Park J, Koltun V. Open3D: A Modern Library for 3D Data Processing[J]. arXiv preprint arXiv:1801.09847, 2018.

[36] Huang G, Liu Z, Maaten L V D, et al. Densely Connected Convolutional Networks[C]//Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017: 2261-2269.

[37] Eldar Y, Lindenbaum M, Porat M, et al. The farthest point strategy for progressive image sampling[J]. IEEE Transactions on Image Processing, 1997, 6(9): 1305-1315.

[38] Bridson R. Fast Poisson disk sampling in arbitrary dimensions[C]//ACM SIGGRAPH 2007 sketches, 2007: 22.

[39] Fan H, Su H, Guibas L. A Point Set Generation Network for 3D Object Reconstruction from a Single Image[C]//Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017: 2463-2471.

[40] Yifan W, Serena F, Wu S, et al. Differentiable surface splatting for point-based geometry processing[J]. ACM Trans Graph, 2019, 38(6): Article 230.

[41] Yu L Q, Li X Z, Fu C W, et al. EC-Net: An Edge-Aware Point Set Consolidation Network[C]//Proceedings of the 15th European Conference on Computer Vision (ECCV), 2018, 11211: 398-414.

[42] Qian G C, Abualshour A, Li G H, et al. PU-GCN: Point Cloud Upsampling using Graph Convolutional Networks[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE I C S C V F, 2021: 11678-11687.

[43] Rubner Y, Tomasi C, Guibas L J. The Earth Mover's Distance as a metric for image retrieval[J]. International Journal of Computer Vision, 2000, 40(2): 99-121.

[44] Sun G X, Wang X C. Three-Dimensional Point Cloud Reconstruction and Morphology Measurement

Method for Greenhouse Plants Based on the Kinect Sensor Self-Calibration[J]. Agronomy-Basel, 2019, 9(10).

[45] Metzer G, Hanocka R, Giryes R, et al. Self-Sampling for Neural Point Cloud Consolidation[J]. ACM Trans Graph, 2021, 40(5): Article 191.

[46] Lee C H, Varshney A, Jacobs D W. Mesh saliency[J]. Acm Transactions on Graphics, 2005, 24(3): 659-666.

[47] Ding X, Lin W, Chen Z, et al. Point Cloud Saliency Detection by Local and Global Feature Fusion[J]. IEEE Transactions on Image Processing, 2019, 28(11): 5379-5393.

[48] Hackel T, Wegner J D, Schindler K. Contour Detection in Unstructured 3D Point Clouds[C]//Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016: 1610-1618.

[49] Wang X, Xu Y, Xu K, et al. PIE-NET: Parametric inference of point cloud edges[C]//Proceedings of the: Neural information processing systems foundation, 2020, 2020-December.

[50] Xu Y Z, Zhu W J, Xu Y L, et al. Dynamic Point Cloud Geometry Compression via Patch-wise Polynomial Fitting[C]//Proceedings of the 44th IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), INST E, ELECT ENGINEERS I E, ELECT ENGINEERS SIGNAL PROC S, 2019: 2287-2291.

[51] Hu S M, Cai J X, Lai Y K. Semantic labeling and instance segmentation of 3d point clouds using patch context analysis and multiscale processing[J]. IEEE Transactions on Visualization and Computer Graphics, 2020, 26(7): 2485-2498.